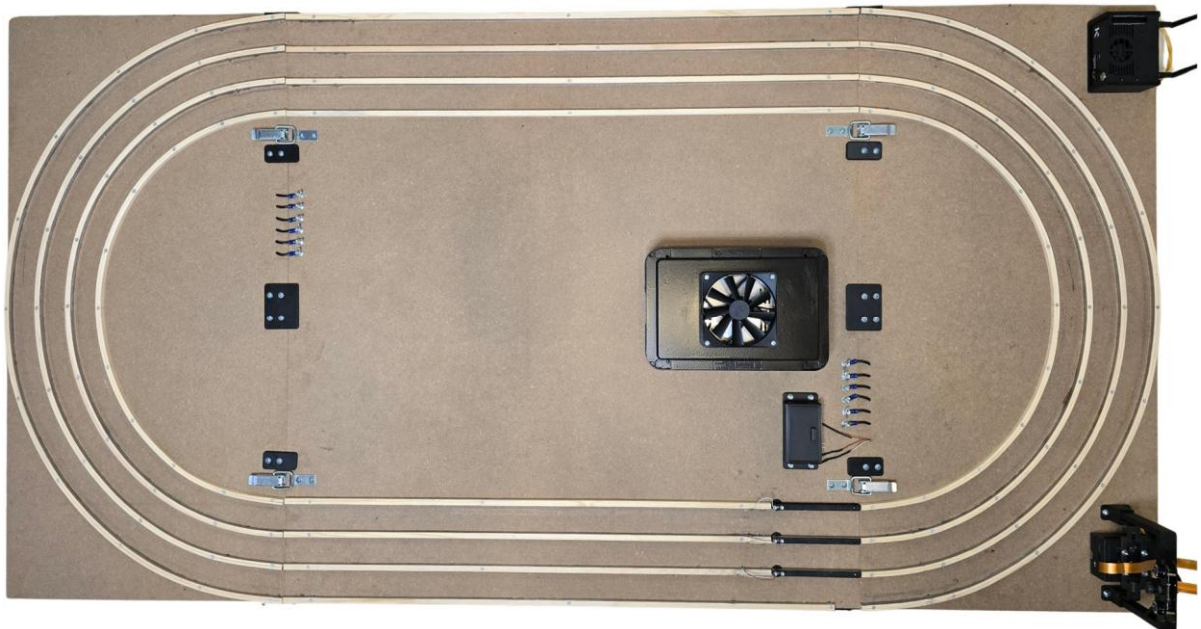




AI-Driven Traffic. Smarter Roads. Smoother Flow.

Teamname: Synapse Mobility

Projektname: LanePilot



Inhalt

Teampräsentation	4
Darstellung der Projektidee	5
a) Zentrale Zielsetzung und Fragestellung	5
b) Von der ersten Idee zur finalen Vision	5
c) LanePilot: Intelligenter Verkehr durch KI	5
d) Unsere Lösung: innovativ und praxisnah	6
Präsentation unserer Lösung	7
a) Mechanik	7
b) Elektrik	13
c) Programmierung	14
Projekt: Gesamtarchitektur und Struktur	15
Datenerfassung, -annotation und -aufbereitung	16
Modellarchitektur und Trainingspipeline	18
Trainingspipeline & Optimierungsstrategien	20
Inferenz, Deployment & Laufzeitsystem	21
Besondere Herausforderungen & Learnings	21
Soziale Auswirkungen & Innovation	22
a) Direkte Auswirkungen auf Menschen in unserer Umgebung	22
b) Potenzielle Auswirkungen auf die globale Bevölkerung	23
c) Gesellschaftlicher Nutzen durch Technologieakzeptanz	23
d) Mögliche Risiken und ethische Fragen	24
e) Innovationsgrad und gesellschaftlicher Fortschritt	24
f) Ausblick: Ein Schritt Richtung Zukunft	25
Weitere nicht-wissenschaftliche Details (Programmcode, Bilder, Making of):	25
Literaturverzeichnis	26
Abkürzungsverzeichnis	29
Anhang	31
Innovations- und Unternehmensaspekte	31
Abbildungen	32
Abb. 1 „Modellbau“	32
Abb. 2 „Motor-Getriebe-Einheit“	32
Abb. 3 „Leitplanken“	33
Abb. 4 „Netzteil & Stromversorgung“	33
Abb. 5.1 „Rad – Version 01“	34
Abb. 5.2 „Rad – Version 05“	34

Abb. 6 „Display – Visualisierung der Entscheidungen“	35
Abb. 7 „Schiebemechanismus“	35
Abb. 8 „Schaltplan“	36
Abb. 9 „Schaubild – Nachrichtentechnik“	36
Abb. 10 „Fahrzeugerkennung mittels YOLOv11“	37
Abb. 11 „Beispielhafte graphische Darstellung eines Verkehrsszenarios“	37
Abb. 12 „Graph Attention Network – Modell-Parameter“	38

Teampräsentation

Wir sind das Team Synapse Mobility und setzen uns aus zwei Mitgliedern im Alter von 17 beziehungsweise 18 Jahren zusammen. Wir besuchen die Jahrgangsstufe 12 der Waldschule Degerloch und haben ein ausgeprägtes Interesse an technischen Themen, insbesondere in den Bereichen Robotik und Künstliche Intelligenz. Darüber hinaus verbindet uns eine gemeinsame Begeisterung für deutsche Automobiltechnik.

Unabhängig davon, ob es sich um Sportwagen, Klassiker oder moderne Konzepte der Elektromobilität handelt, schätzen wir deutsche Automobilmarken und deren ingenieurtechnische Qualität in besonderem Maße. Diese Faszination diente auch als zentrale Inspirationsquelle bei der Entwicklung von „LanePilot“. Analog zur Automobilindustrie verfolgten wir das Ziel, Präzision, Innovationskraft und Zuverlässigkeit in einem modernen Verkehrssystem miteinander zu vereinen.

- Kaan Gönüldinc (Konstruktion, Recherche, Elektrik, Programmierung, Dokumentation)
- Ferdinand Helber (Konstruktion, Recherche, Modellbau, Dokumentation)
- Emil Jurich, Coach (Betreuung des Projektes)



(v. l. n. r.: Ferdinand Helber, Kaan Gönüldinc, Emil Jurich)

Darstellung der Projektidee

a) Zentrale Zielsetzung und Fragestellung

Ziel dieser besonderen Lernleistung ist es, zu untersuchen, inwiefern eine KI-basierte, spurdynamische Verkehrssteuerung durch intelligente Fahrzeugzuweisung den Verkehrsfluss effizienter gestalten kann und wie sich ein solcher Ansatz realitätsnah modellieren, simulieren und evaluieren lässt.

b) Von der ersten Idee zur finalen Vision

Zu Beginn unserer Projektarbeit setzten wir uns mit unterschiedlichen Anwendungsfeldern auseinander, in denen robotische Systeme einen konkreten gesellschaftlichen Nutzen leisten könnten. Eine der ersten im Team diskutierten Ideen war die Konzeption eines Hilfs-, Notfall- oder Bergungsroboters. Ausgangspunkt dieser Überlegung war die Möglichkeit, Menschen in gefährlichen oder unzugänglichen Situationen durch autonome oder ferngesteuerte Systeme zu unterstützen, etwa beim Erreichen schwer zugänglicher Orte, bei der Suche nach vermissten Personen oder bei der Versorgung Betroffener in Katastrophengebieten.

Im Zuge einer umfassenden Internetrecherche wurde jedoch deutlich, dass in diesem Bereich bereits eine Vielzahl ausgereifter Lösungen existiert, sowohl im zivilen als auch im militärischen Einsatz. Dazu zählen unter anderem kettengetriebene Roboter zur Trümmererkundung, Drohnen mit Wärmebildsensoren sowie Systeme mit multifunktionalen Roboterarmen. Solche Technologien werden bereits heute bei Naturkatastrophen wie Erdbeben oder Überschwemmungen, aber auch in Krisenregionen und bei spezialisierten Rettungseinsätzen eingesetzt.

Vor diesem Hintergrund unterzogen wir die ursprüngliche Idee einer kritischen Bewertung. Einerseits erschien der Innovationsgrad unseres Konzepts im Vergleich zu bestehenden Systemen als gering. Andererseits hätte die praktische Umsetzung voraussichtlich zur Verwendung von bereits verfügbaren Technologien, Modulen oder Bauteilen geführt. Dies widersprach unserem Anspruch, eine eigenständige, innovative und vollständig selbst entwickelte Lösung zu realisieren.

Diese Einsicht stellte einen bedeutenden Wendepunkt in unserer Projektarbeit dar. Anstatt eine bereits etablierte Anwendung nachzubilden, entschieden wir uns bewusst dafür, ein gesellschaftlich relevantes Problem, das bislang nur unzureichend gelöst ist, neu zu adressieren und ein eigenes Modell zu entwickeln. Auf dieser Grundlage entstand das Projekt LanePilot.

c) LanePilot: Intelligenter Verkehr durch KI

Unser Projekt LanePilot beschäftigt sich mit einem der drängendsten Probleme urbaner Mobilität: Staus, ungleichmäßige Fahrbahnauslastung und ineffizienter Verkehrsfluss. Täglich verlieren Menschen weltweit wertvolle Zeit im Verkehr, verursachen durch unnötiges Stop-and-Go hohe CO₂-Emissionen und setzen sich einem erhöhten Unfallrisiko aus. Eine Vielzahl dieser Probleme resultiert nicht aus mangelnder Infrastruktur, sondern aus ineffizienter Nutzung vorhandener Spuren.

In vielen Fällen werden Verkehrssysteme lediglich reaktiv angepasst, etwa durch Umleitungen bei Baustellen oder durch elektronische Geschwindigkeitstafeln, anstatt das Verkehrsverhalten proaktiv und intelligent zu steuern. Bestehende Lösungsansätze wie Navigationsanwendungen oder statische Verkehrszeichen greifen nur eingeschränkt in den Verkehrsfluss ein. Sie informieren über Staus oder schlagen alternative Routen vor, beeinflussen jedoch nicht aktiv die Echtzeitznutzung einzelner Fahrspuren.

An dieser Stelle setzt LanePilot an. Mithilfe moderner Verfahren unter Nutzung von Künstlicher Intelligenz analysiert das System die aktuelle Verkehrssituation in Echtzeit und berechnet Spurzuweisungen für einzelne Fahrzeuge. Grundlage hierfür sind Bilddaten, die durch ein neuronales Netz zur Segmentierung (YOLOv11-Segmentation¹) ausgewertet werden, sowie ein eigens entwickeltes Deep-Learning-Modell, das optimale Spurverteilungen ermittelt und im [entsprechenden Abschnitt](#) dieser Arbeit näher erläutert wird. Dieses Modell bildet das Kernelement des Gesamtsystems, da es nicht nur Fahrzeuge erkennt, sondern deren optimale Positionierung innerhalb des Verkehrsflusses bestimmt und die Entscheidung darüber trifft, welche Fahrspur für welches Fahrzeug unter den gegebenen Bedingungen am geeignetsten ist.

Die Relevanz von LanePilot geht über den klassischen Straßenverkehr hinaus. Vor dem Hintergrund zunehmender urbaner Verdichtung, wachsender Umweltbelastungen und eines steigenden Automatisierungsgrades bei Fahrzeugen, bietet das System einen praxisnahen und skalierbaren Lösungsansatz für Smart Cities, öffentliche Institutionen und Mobilitätsdienstleister. Durch eine effizientere Nutzung der vorhandenen Infrastruktur trägt LanePilot zur Reduktion von Staus, zur Verbesserung der Luftqualität und zur Erhöhung der Verkehrssicherheit bei.

d) Unsere Lösung: innovativ und praxisnah

Anstelle vordefinierter Regeln oder starrer Entscheidungslogiken basiert das System auf einem lernenden Ansatz. Durch die Auswertung von Trainingsdaten und einer Umgebungssimulation mittels Reinforcement-Learning (RL) erlernt das Modell, welche Spur unter bestimmten Rahmenbedingungen, etwa in Abhängigkeit von Verkehrsdichte, Fahrzeugabständen und Geschwindigkeiten, als optimal einzustufen ist. Auf diese Weise kann das System flexibel auf unterschiedliche Verkehrssituationen reagieren und seine Leistungsfähigkeit im Laufe der Zeit weiter verbessern.

Darüber hinaus wurde die Interaktion mit den Verkehrsteilnehmern konzeptionell berücksichtigt. Die berechneten Spurzuweisungen können zukünftig entweder über intelligente Anzeigetafeln oder direkt mittels V2I²-Kommunikation an die Fahrzeuge übermittelt werden. Dadurch ist LanePilot

¹ **YOLOv11-Segmentation:** Die elfte Version der Computer-Vision-Modellreihe von Ultralytics Inc., die zur Bilderkennung und Instanzsegmentierung dient. Im Gegensatz zu den Vorgängerversionen bietet diese Version verbesserte Segmentierungsgenauigkeit bei geringer Latenz.

² **Vehicle-To-Infrastructure (V2I):** Kommunikationsweg, bei dem ein Fahrzeug Daten mit der umgebenden Infrastruktur (Ampeln, Signalanlagen, Verkehrsleitzentralen, usw.) austauscht.

sowohl mit aktuellen Fahrzeugkonzepten als auch mit zukünftigen Entwicklungen im Bereich der vernetzten Mobilität kompatibel.

Die V2X³- beziehungsweise V2I-Kommunikation wird bereits in mehreren deutschen Städten erprobt und teilweise erfolgreich eingesetzt. In Hamburg wurden beispielsweise mehr als 140 Ampelanlagen mit intelligenter Kommunikationstechnologie ausgestattet, um Fahrzeuge mit Informationen zu Ampelphasen und Verkehrsfluss zu versorgen. Auch in Merzig im Saarland wurde eine Plattform geschaffen, auf der Anwendungen wie Ampelassistenten oder Baustellenwarnsysteme getestet werden. Diese Beispiele verdeutlichen, dass die notwendige Infrastruktur für vernetzte Mobilität zunehmend verfügbar ist. LanePilot lässt sich nahtlos in solche V2I-Umgebungen integrieren und kann durch den bidirektionalen Austausch von Daten zwischen Fahrzeugen und Infrastruktur sowohl die Verkehrssicherheit als auch die Effizienz des Straßenverkehrs deutlich erhöhen.

Die Anwendbarkeit der entwickelten Lösung beschränkt sich nicht auf den urbanen Raum. Auch auf Autobahnen, in Industriegebieten oder bei Großveranstaltungen mit hohem Verkehrsaufkommen kann LanePilot eingesetzt werden. In allen Situationen, in denen eine hohe Fahrzeugdichte vorliegt und ein effizienter Verkehrsfluss von enormer Bedeutung ist, bietet das System das Potenzial, spürbare Verbesserungen zu erzielen.

Präsentation unserer Lösung

a) Mechanik

Ziel der Konstruktion war die Entwicklung einer anschaulichen und zugleich technisch handhabbaren Simulationsumgebung zur Demonstration des LanePilot-Konzepts. In einem für Wettbewerbe und Präsentationen geeigneten Maßstab wurde eine eigenständige Modellanlage konzipiert und realisiert, die einen mehrspurigen Straßenabschnitt mit kontinuierlichem Verkehrsfluss sowie der Möglichkeit zu Spurwechseln nachbildet.

Das Ergebnis ist eine ovale Modellstrecke mit drei Fahrspuren, die über integrierte Leitplanken zur Spurführung, eine zentrale Energieversorgung, fahrzeugähnliche Module und eine intelligente Steuereinheit verfügt. Diese Anlage ermöglicht es, die wesentlichen Aspekte der KI-basierten Spurverteilung anschaulich darzustellen und deren Wirkungsweise visuell nachvollziehbar zu machen (siehe [Abb. 1 „Modellbau“](#)).

Zu Demonstrationszwecken war vorgesehen, die fahrzeugähnlichen Module zunächst bewusst ungleichmäßig auf die Fahrspuren zu verteilen. Dabei sollte eine hohe Fahrzeugdichte auf der inneren Spur, eine geringere Belegung der mittleren Spur und eine nahezu ungenutzte äußere Spur entstehen. Das System hätte diese unausgeglichene Auslastung automatisch erkannt und über mehrere Umläufe hinweg gezielte Weichenstellungen vorgenommen. Auf diese Weise wären die

³ **Vehicle-To-Everything (V2X)**: Sammelbegriff, der sowohl die Kommunikation mit anderen Verkehrsteilnehmern (V2V) als auch V2I miteinschließt.

Fahrzeuge schrittweise gleichmäßig auf alle drei Spuren verteilt worden. Dadurch hätte sich unmittelbar am Modell zeigen lassen, wie LanePilot aktiv in die Spurverteilung eingreift und zur Steigerung der Verkehrseffizienz beiträgt.

Die gesamte Modellanlage wurde auf eine maximale Grundfläche von 2 x 1 Metern begrenzt. Diese Abmessung stellte von Beginn an die bauliche Randbedingung dar, da sie sowohl den Wettbewerbsvorgaben als auch den Anforderungen an Transport und Aufbau entsprechen musste. Die Fahrbahn ist als Rundstrecke ausgeführt, um einen kontinuierlichen Verkehrsfluss zu simulieren.

Als Untergrund dient eine stabile Holzplatte mit einer Stärke von 12 Millimetern, die sich hinsichtlich Biegefestigkeit und Bearbeitbarkeit als besonders geeignet erwies. Die drei Fahrspuren verlaufen parallel nebeneinander und sind durch fest installierte Leitplanken voneinander getrennt. Diese Leitplanken übernehmen nicht nur die visuelle Abgrenzung der Spuren, sondern integrieren zugleich elektrisch leitende Elemente, über die die Fahrzeuge mit Energie versorgt werden. Die ursprüngliche Idee, handelsübliche und batteriebetriebene Modellautos zu verwenden, wurde aus ökologischen Gründen und aufgrund der begrenzten Auswahl verworfen. Deshalb wurde ein eigenes Fahrzeugkonzept entwickelt.

Die im Modell eingesetzten Fahrzeuge bestehen aus kompakten Motor-Getriebe-Einheiten (siehe [Abb. 2 „Motor-Getriebe-Einheit“](#)), die um notwendige Komponenten wie Räder und Stromabnehmer ergänzt wurden. Ziel dieser Konstruktion war eine bewusst minimalistische Auslegung, die sich auf die für den Betrieb relevanten Funktionen beschränkt. Die Energieversorgung erfolgt über Schleifkontakte, welche den Strom direkt von den in den Leitplanken integrierten Leiterbahnen aufnehmen. Dieses Prinzip ist mit klassischen Systemen wie Carrera-Rennbahnen oder Modelleisenbahnen vergleichbar, wurde jedoch technisch erweitert, um variable Geschwindigkeiten sowie gezielte Spurwechsel zu ermöglichen.

Zur Erzeugung unterschiedlicher Fahrgeschwindigkeiten wurden bei ausgewählten Fahrzeugen gezielt Widerstände in die Stromversorgung integriert. Dadurch bewegen sich einzelne Fahrzeuge deutlich langsamer als andere. Diese Form der Geschwindigkeitsregulation erlaubt es, im Modell realitätsnahe Verkehrssituationen nachzustellen, etwa langsam fahrende Fahrzeuge auf Überholspuren. Solche Szenarien stellen eine wesentliche Grundlage für die Funktionsweise der entwickelten Spurwechsellogik dar.

Am Ende des geraden Streckenabschnitts wurde eine Brückenkonstruktion installiert, auf der eine hochauflösende Kamera montiert ist. Diese erfasst gleichzeitig die Fahrzeuge auf allen drei Spuren und ermittelt deren Positionen sowie Geschwindigkeiten. Auf Basis dieser Echtzeitdaten würden im realen Einsatz Spurwechsel-Empfehlungen beispielsweise über Anzeigetafeln, Projektionen auf die Fahrbahn oder V2I-Kommunikation ausgegeben. In der Modellanlage wird diese Funktion durch eine automatisierte Weichensteuerung am Ende des geraden Abschnitts umgesetzt.

Langsam fahrende Fahrzeuge auf der schnellen, inneren Fahrbahn werden automatisch auf die mittlere Spur umgeleitet. Entsprechend werden langsame Fahrzeuge auf der mittleren Spur auf die rechte Spur verschoben. Dieses Vorgehen verdeutlicht anschaulich, wie LanePilot dazu beitragen kann, Behinderungen auf schnelleren Spuren zu vermeiden und den Verkehrsfluss zu optimieren.

Die innere Spur des ovalen Streckenverlaufs wurde bewusst als schnelle Spur festgelegt. Bei einer umgekehrten Anordnung hätte sich ein unerwünschter Effekt ergeben, da langsam fahrende Fahrzeuge auf der kurveninneren Spur aufgrund der kürzeren Wegstrecke potenziell schnellere Fahrzeuge auf den äußeren Spuren hätten überholen können. Dies hätte zu unrealistischen Abläufen innerhalb des Modells geführt, die nicht dem realen Verkehrsverhalten entsprechen.

Um einen praktikablen Transport der Modellanlage zu ermöglichen, wurde diese modular konzipiert. Sie besteht aus drei Hauptsegmenten, einem mittleren Element mit geraden Strecken sowie zwei Endsegmenten mit gekrümmten Fahrbahnabschnitten. Diese Module sind so ausgeführt, dass sie sich mechanisch präzise miteinander verbinden und bei Bedarf wieder trennen lassen. Eine bedeutsame Herausforderung bestand darin, die einzelnen Fahrspuren an den Übergängen der Module exakt fluchtend auszurichten, da bereits geringfügige Versätze dazu führen konnten, dass Fahrzeuge an den Stoßstellen hängen bleiben. Eine genaue Ausrichtung war daher entscheidend für einen störungsfreien Betrieb der gesamten Anlage.

Für die Herstellung der Leitplanken wurde zunächst auf handelsübliche Holzleisten zurückgegriffen, die in unterschiedlichen Abmessungen kostengünstig verfügbar waren. Um eine stabile Verschraubung auf der Holzgrundplatte zu gewährleisten, war eine bestimmte Materialstärke erforderlich. Während sich dies bei den geraden Streckenabschnitten problemlos realisieren ließ, traten bei den gekrümmten Bereichen erhebliche Schwierigkeiten auf. Die vergleichsweise dicken Holzleisten erwiesen sich als nur schwer biegsam und ließen sich nicht ohne Weiteres in die gewünschte Form bringen.

Zur Lösung dieses Problems wurden für die Kurvenabschnitte zwei schmale Holzleisten verwendet, die sich deutlich leichter verformen ließen (siehe [Abb. 3 „Leitplanken“](#)). Nach dem Biegen wurden diese miteinander verklebt, um die erforderliche Gesamtbreite für eine stabile Befestigung auf der Grundplatte zu erreichen. Die Fertigung dieser gebogenen Leitplanken war jedoch mit einem hohen Aufwand verbunden. Während des Verleimprozesses mussten die Leisten exakt im vorgesehenen Radius fixiert werden. Zu diesem Zweck wurde eine spezielle Schablone angefertigt, die die Bauteile während des Trocknens in der gewünschten Geometrie hielt. Rückblickend wäre es effizienter gewesen, die Leitplanken der gekrümmten Abschnitte in einzelne Segmente zu unterteilen und diese mittels 3D-Druck herzustellen. Dadurch hätte sich der Fertigungsaufwand deutlich reduzieren lassen, bei gleichzeitig höherer Maßhaltigkeit.

Im Anschluss wurden auf den Holzleisten Leiterbahnen aus Nickel angebracht, die der Stromversorgung der Fahrzeuge dienen. Jede Fahrspur verfügt über einen eigenen Stromkreis, wobei alle Kreise von einem gemeinsamen Netzteil gespeist werden (siehe [Abb. 4 „Netzteil &](#)

[Stromversorgung](#)“). Diese bewusste Trennung der Fahrstromkreise ermöglicht unterschiedliche Bestromungen der Spuren, um unterschiedliche Fahrzeuggeschwindigkeiten zu erzielen.

Die Brückenkonstruktion am Ende des geraden Streckenabschnitts wurde ebenfalls demontierbar ausgeführt, um den Transport der Anlage zu erleichtern und Beschädigungen zu vermeiden. Gleichzeitig musste sie eine ausreichende Höhe aufweisen, um den gesamten geraden Streckenabschnitt vollständig erfassen zu können.

Auf der Brückenkonstruktion ist eine hochauflösende Kamera installiert, die mit einem kompakten Rechnersystem verbunden ist. Dieses übernimmt die Bildverarbeitung mithilfe des zuvor trainierten YOLOv11-Segmentierungsmodells (vgl. [Programmierung](#)). Die Kamera erfasst gleichzeitig alle drei Fahrspuren, erkennt die Fahrzeuge, bestimmt deren Positionen sowie Geschwindigkeiten und übermittelt diese Informationen an das zentrale Steuerungssystem.

Innerhalb des Erfassungsbereichs der Kamera befinden sich zudem die elektromechanischen Weichen, welche die Fahrzeuge gezielt auf andere Fahrspuren lenken können. Dabei handelt es sich um kurze, schwenkbare Leitplankenelemente, die über Servomotoren angesteuert werden und so einen kontrollierten Spurwechsel ermöglichen.

Die ersten Testläufe auf der vollständig montierten Modellanlage verliefen weniger erfolgreich als erwartet. Trotz sorgfältiger Planung und präziser Montage zeigte sich rasch, dass zwischen theoretischem Konzept und praktischer Umsetzung deutliche Unterschiede bestanden. Wiederholt blieben Fahrzeuge stehen oder verließen in den Kurvenbereichen die vorgesehene Spurführung. Ein besonders kritischer Punkt stellte dabei die kontinuierliche Stromversorgung dar, da zahlreiche Fahrzeuge in bestimmten Streckenabschnitten kurzzeitig den Kontakt zu den Leiterbahnen verloren.

Obwohl der Abstand der Leitplanken in den Kurven identisch zu dem der geraden Streckenabschnitte war, traten dort regelmäßig mechanische Blockaden auf. Die Ursache lag in der ursprünglichen Auslegung der Fahrzeuge, die für eine geradlinige Fahrbewegung optimiert war. In Kurven benötigten die Fahrzeuge jedoch mehr Spurbreite, was zu Kollisionen mit den Leitplanken und wiederholten Entgleisungen führte.

Ein weiteres relevantes Problem betraf die Haftung der Fahrzeugräder auf der Fahrbahn. Die glatte Holzoberfläche bot nicht ausreichend Reibung, wodurch die Räder durchdrehten und die Fahrzeuge teilweise nicht mehr zuverlässig angetrieben werden konnten. Da die gesamte Funktionsweise des Systems auf der Simulation eines kontinuierlichen Fahrzeugflusses mit unterschiedlichen Geschwindigkeiten auf drei Spuren basiert, gefährdete dieser Mangel die Aussagekraft des gesamten Modells.

Infolgedessen wurde die Ausführung der Räder grundlegend überarbeitet. Die anfängliche, kostengünstige Lösung mit großformatigen Unterlegscheiben (siehe [Abb. 5.1 „Rad – Version 01“](#)) wurde verworfen und durch breitere, 3D-gedruckte Räder ersetzt. Diese wurden als Hohlkörper

konstruiert: Zur Verbesserung der Traktion wurde der Hohlraum mit metallischen Unterlegscheiben gefüllt, wodurch das Gewicht gezielt auf die Antriebsachse verlagert und die Bodenhaftung der Fahrzeuge deutlich erhöht wurde.

Trotz dieser zusätzlichen Maßnahme erwies sich der Grip zunächst weiterhin als unzureichend. Aus diesem Grund wurden die Laufflächen der Räder zusätzlich mit Schleifpapier beklebt. Durch die raue Oberfläche konnte die Reibung deutlich erhöht und die Traktion spürbar verbessert werden, was sich unmittelbar positiv auf das Anfahrverhalten und die Stabilität während der Fahrt auswirkte.

Um in den Kurvenbereichen mehr seitlichen Freiraum zu schaffen, wurden die Räder nachträglich in ihrer Breite reduziert und ihre Kanten sorgfältig verrundet. Dadurch konnten sie besser entlang der Leitplanken gleiten, ohne zu verkanten oder die Spur zu verlassen. Kaum ein Bauteil des Systems wurde im Verlauf des Projekts so häufig angepasst und optimiert wie die Räder, was deren Bedeutung für die Gesamtfunktionalität der Anlage aufzeigt (siehe [Abb. 5.2 „Rad – Version 05“](#)).

Bereits zu Beginn des Projekts wurde der Mechanismus für den Spurwechsel intensiv diskutiert. In Betracht gezogen wurden unter anderem Förderbandsysteme mit variabler Laufrichtung, wie sie aus der Logistik bei Paketverteilzentren bekannt sind. Ebenso wurden Linearschieber evaluiert, die Fahrzeuge mechanisch auf eine andere Spur verschieben könnten, sowie der Einsatz gezielter Luftströme, wie sie in industriellen Sortieranlagen zur Mülltrennung Verwendung finden.

Letztlich entschieden wir uns für ein Weichenkonzept, das eine gezielte Umlenkung der Fahrzeuge ermöglicht. Im Vergleich zu Luftströmen oder einem mechanischen Stoß durch einen „Stößel“ erschien diese Lösung aus unserer Sicht eleganter, präziser und besser kontrollierbar. Die Umsetzung erwies sich jedoch als problematisch, da die gestellten Weichen die effektive Fahrbahnbreite reduzierten. Fahrzeuge, die sich auf der betroffenen Spur befanden, konnten die entstehenden Engstellen nicht mehr passieren, was zu Blockaden und Stillständen führte.

Da im weiteren Projektverlauf keine ausreichende Zeit mehr zur Verfügung stand, um den Mechanismus grundlegend umzubauen, wurde eine alternative Lösung realisiert. Hierbei kam ein Display zum Einsatz, das analog zu realen Verkehrsanzeigetafeln genutzt wurde. Über diese visuelle Schnittstelle konnten für alle drei Spuren situationsabhängige Anweisungen ausgegeben werden. Auf diese Weise ließ sich die Funktionsweise des Algorithmus dennoch anschaulich demonstrieren, insbesondere das Zusammenspiel aus Kameraerfassung, Bildauswertung und anschließender Befehlsausgabe an das System (siehe [Abb. 6 „Display – Visualisierung der Entscheidungen“](#)).

Unabhängig davon entwickelten wir für den Spurwechselmechanismus eine alternative, konzeptionell funktionsfähige Lösung. Der folgende Mechanismus stellt eine technisch vollständig ausgearbeitete, jedoch im Rahmen dieser Projektphase nicht mehr realisierte Lösung dar, die als Grundlage für zukünftige Weiterentwicklungen dient. Die Herangehensweise basiert auf einer Schiebepalte, die in die Bodenstruktur integriert ist und Teile der Leitplanke einschließlich der Leiterbahnen enthält (siehe [Abb. 7 „Schiebemechanismus“](#)). Durch eine seitliche Verschiebung

dieser Platte können Fahrzeuge parallel zur Fahrbahn kontrolliert auf eine benachbarte Spur bewegt werden, ohne die verfügbare Fahrbahnbreite einzuschränken. Damit wird das Problem der bisherigen Weichenlösung vermieden und ein deutlich robusterer Ansatz für eine zukünftige Version des Systems geschaffen.

Für diese Bauweise ist vorgesehen, das System um eine zusätzliche, vierte Spur zu erweitern, die ausschließlich während des eigentlichen Verschiebevorgangs genutzt wird. Diese Spur dient somit nicht dem regulären Fahrbetrieb, sondern fungiert als temporäre Übergangsspur für den Spurwechsel. Der zentrale Vorteil dieses Konzepts besteht darin, dass die Schiebeplatte nach einem vollzogenen Spurwechsel nicht sofort in ihre Ausgangsposition zurückkehren muss. Stattdessen kann sie in der verschobenen Position verbleiben, bis eine geeignete Verkehrssituation eine Rückführung erlaubt. Dadurch wird die mechanische Belastung des Systems verringert und der Durchsatz an möglichen Spurwechseln erhöht. Gleichzeitig könnte sich auf diese Weise die Gesamtverstellzeit des Mechanismus im Idealfall halbieren.

Konstruktiv ist die Schiebeplatte auf zwei Linearschienen montiert. Diese Schienen sind mit kugelgelagerten Laufwagen ausgestattet und ermöglichen eine spielfreie, hochpräzise und leichtgängige Querbewegung der gesamten Baugruppe. Die parallele Anordnung der beiden Schienen sorgt für eine stabile Führung, verhindert Verkanten und gewährleistet auch bei höheren Beschleunigungen eine gleichmäßige Kraftverteilung.

Der Antrieb der Schiebeplatte erfolgt über einen Zahnriemen, der fest mit der Platte verbunden ist und entlang der vorgesehenen Bewegungslinie gespannt verläuft. Dieser Zahnriemen wird von einem Schrittmotor angetrieben, der schnelle und zugleich präzise Positionsänderungen auch unter Last zuverlässig ausführen kann. Die auf der Motorwelle montierte Zahnriemenscheibe greift formschlüssig in den Riemen ein, wodurch ein ruckfreier Lauf sowie eine hohe Wiederholgenauigkeit der Bewegung sichergestellt werden.

Zur Positionsüberwachung der Schiebeplatte sind Endschalter oder alternativ Hall-Sensoren vorgesehen. Diese ermöglichen beim Systemstart eine automatische Referenzfahrt, bei der eine definierte Nullposition angefahren wird. Gleichzeitig verhindern sie ein Überfahren der mechanischen Endlagen und erhöhen somit die Betriebssicherheit und Lebensdauer des Systems.

Die übergeordnete Steuerung der Bewegung erfolgt über einen Mikrocontroller, beispielsweise einen Arduino oder einen Raspberry Pi. Dieser verarbeitet externe Sensordaten, insbesondere die Informationen aus der kamerabasierten Fahrzeugerkennung, und entscheidet situationsabhängig, ob ein Spurwechsel eingeleitet werden soll. Sobald ein Fahrzeug im vorgesehenen Zielbereich erkannt wird und die benachbarte Spur als frei identifiziert ist, aktiviert die Steuerung den Schrittmotor. Die Schiebeplatte wird daraufhin seitlich verfahren und nimmt das Fahrzeug während der Bewegung physisch mit auf die neue Spur. Auf diese Weise wird ein kontrollierter, reproduzierbarer und verkehrssicherer Spurwechsel realisiert, der ohne Verengung der Fahrbahnbreite auskommt und sich gut in das Gesamtkonzept von LanePilot integrieren lässt.

b) Elektrik

Die zentrale Steuereinheit des Gesamtsystems bildet ein Raspberry Pi 5 mit 8 GB Arbeitsspeicher (RAM). Dieses System fungiert als übergeordneter Kommunikationsknoten und vernetzt sämtliche relevanten Hardware- und Softwarekomponenten miteinander. An den Raspberry Pi ist unter anderem eine infrarotfähige Full-HD-Kamera angeschlossen. Die von dieser Kamera erfassten Videodaten werden in Echtzeit über eine kabelgebundene Ethernet-Verbindung an den NVIDIA Jetson Orin Nano Super mit 8 GB RAM übertragen, der als dedizierte KI-Recheneinheit eingesetzt wird. Neben der Erfassung und Weiterleitung der Bilddaten übernimmt der Raspberry Pi zudem die direkte Ansteuerung der Servomotoren, welche für die Umsetzung der vom KI-System berechneten Steuerbefehle verantwortlich sind.

Darüber hinaus ist der Raspberry Pi für die visuelle Aufbereitung der vom Jetson berechneten Handlungsempfehlungen zuständig. Diese Empfehlungen werden in Form einfacher grafischer Darstellungen, beispielsweise Richtungspfeilen oder Spurhinweisen, aufbereitet und in statische Bilddateien umgewandelt (*vgl. vorherige Abbildungen*). Die Übertragung dieser Visualisierungen erfolgt über einen integrierten Access Point, der direkt vom Raspberry Pi bereitgestellt wird. Über diesen lokalen WLAN-Zugang werden die aktuellen Systeminformationen mittels einem Livestream-Server zur Verfügung gestellt, wodurch mobile Endgeräte wie Tablets oder Laptops ohne zusätzliche Infrastruktur auf den aktuellen Systemzustand zugreifen können.

Die eigentliche KI-basierte Inferenz findet jedoch vollständig auf dem NVIDIA Jetson Orin Nano Super statt, der innerhalb des Gesamtsystems die Rolle der zentralen Verarbeitungseinheit übernimmt. Die vom Raspberry Pi gelieferten Bilddaten werden mithilfe der integrierten NVIDIA-GPU verarbeitet. Die eingesetzten neuronalen Netze analysieren auf dieser Grundlage die aktuelle Verkehrssituation, identifizieren Fahrzeuge, bestimmen deren Positionen und Geschwindigkeiten und berechnen anschließend eine optimale Spuruweisung. Die daraus resultierenden Steueranweisungen, beispielsweise die Vorgabe, ein Fahrzeug von einer Spur auf eine benachbarte Spur zu verlagern, werden anschließend an den Raspberry Pi zurückübermittelt und dort weiterverarbeitet.

Für die Ansteuerung der Servomotoren wurde eine eigens entwickelte Kabelbox realisiert, die mehrere elektronische Schaltungen zur Umsetzung des erforderlichen Kommunikationsprotokolls enthält. Im Fokus steht hierbei die Datenübertragung über die UART-Schnittstelle des Raspberry Pi. Da diese Schnittstelle ausschließlich Full-Duplex⁴-Übertragung unterstützt, der Servomotor aber Half-Duplex⁵ benötigt, war der Einsatz eines Konvertierungsmechanismus notwendig. Dabei wird der TX-Pin als Pull-Up-Leitung genutzt und über zwei parallel geschaltete 10kΩ-Widerstände mit dem RX-Pin verbunden. Die daraus resultierende Datenleitung wird direkt in den DATA-Pin eines der Servomotoren eingespeist. Diese selbst entwickelte Full-Duplex-zu-Half-Duplex-Schaltung

⁴ **Full-Duplex:** Bidirektionaler Kommunikationsweg, bei dem Daten zwischen zwei Akteuren simultan in beide Richtungen übertragen werden können.

⁵ **Half-Duplex:** Ein ebenfalls bidirektionaler Kommunikationsweg zur Datenübertragung, jedoch auf nur einer Datenleitung und daher sequenziell (nur in eine Richtung zu einem bestimmten Zeitpunkt).

stellte eine funktionale und zugleich kosteneffiziente Lösung dar und ermöglichte gegenüber kommerziellen Adaptern eine Einsparung von rund 80 Euro.

Die Stromversorgung des Systems erfolgt zentral über ein gemeinsames 12-V-Netzteil. Dessen Ausgangsspannung wird mithilfe eines Step-Down-Konverters auf 7,4V reduziert, was der Nennbetriebsspannung der eingesetzten Servomotoren entspricht. Zur Stabilisierung der Versorgungsspannung wurde parallel zum Ausgang des Konverters ein Elektrolytkondensator mit einer Kapazität von 1000µF geschaltet. Dieser dient dazu, Spannungsschwankungen zu glätten und kurzzeitige Lastspitzen abzufangen. Der Spannungsabgriff erfolgt direkt am Ausgang des Konverters: Die 7,4V versorgen die Servomotoren, während die gemeinsame Masse (*GND*) ebenfalls mit dem Raspberry Pi verbunden ist, um eine saubere gemeinsame Erdung (*Common Ground*) sicherzustellen. An den initialen Servomotor wurden im Daisy-Chaining-Verfahren⁶ zwei weitere Servos desselben Typs angeschlossen, sodass mehrere Aktuatoren effizient über eine einzige Datenleitung angesprochen werden können (siehe [Abb. 8 „Schaltplan“](#) und [Abb. 9 „Schaubild – Nachrichtentechnik“](#)). Die Leiterbahnen, über die die Servomotoren mit Spannung versorgt werden, bestehen aus Nickel, da dieses Material eine ausreichende elektrische Leitfähigkeit bei gleichzeitig guter mechanischer Belastbarkeit bietet, während Silber aus Kostengründen ausschied und Kupfer in der benötigten Form eines dünnen, flexiblen Bandes nicht verfügbar war.

Das Daisy-Chaining über lediglich eine gemeinsame Datenleitung führte jedoch dazu, dass sich die angeschlossenen Servomotoren gegenseitig beeinflussten, da die eigens entwickelte Duplex-Schaltung vergleichsweise einfach aufgebaut und entsprechend störanfällig (z. B. Signalüberlagerungen) war. Dieses Problem konnte erfolgreich beseitigt werden, indem die Baudrate softwareseitig von ursprünglich 1.000.000Bd auf 115.200Bd reduziert wurde.

c) Programmierung⁷

Die mechanische Modellanlage diente ausschließlich als Demonstrations- und Datenerfassungsumgebung. Die eigentliche Systemintelligenz des Projekts entsteht jedoch in der Softwarearchitektur und den KI-Modellen durch die Verarbeitung von Sensordaten, die strukturierte Aufbereitung dieser Informationen sowie durch maschinelle Lernverfahren, die aus diesen Daten fundierte Entscheidungen ableiten. Hardware wie Kameras, Recheneinheiten, Aktoren oder unser physisches Modell dient dabei ausschließlich als Mittel zur Datenerfassung und Ausführung, während die eigentliche Problemlösung vollständig softwaregetrieben erfolgt.

Ziel der Softwareentwicklung war es, einen forschungsnahen Prototypen zu entwerfen, der reale Verkehrsszenarien modellieren, analysieren und optimieren kann. Dabei wurde bewusst auf eine modulare, erweiterbare und reproduzierbare Programmstruktur geachtet, um sowohl zukünftige

⁶ **Daisy-Chaining:** Verdrahtungsschema, bei dem mehrere Geräte, wie z. B. Netzteile oder Motoren, direkt hintereinandergeschaltet werden.

⁷ **Hinweis:** Der gesamte Quellcode inkl. aller relevanten Ressourcen, Dateien und Firmware-Images ist auf der unten verlinkten [GitHub-Repository](#) zu finden und aufgrund der Länge nicht in dieser Arbeit angehängt.

Verbesserungen als auch alternative KI-Ansätze problemlos integrieren zu können. LanePilot ist somit nicht als kurzfristige Demonstration konzipiert, sondern als technisch saubere Grundlage für weiterführende Forschung und Entwicklung im Bereich intelligenter Verkehrssysteme.

Ein weiterer Aspekt der Software ist die Trennung zwischen Offline-Prozessen wie Datengenerierung und KI-Training sowie Online-Prozessen wie Inferenz und Laufzeitverhalten auf eingebetteten, rechennahen Systemen (sog. „Edge-Devices“). Diese klare Abgrenzung ermöglicht es, komplexe Modelle effizient zu trainieren und anschließend ressourcenschonend auf Zielhardware wie dem NVIDIA Jetson oder dem Raspberry Pi einzusetzen.

Zusammenfassend bildet die Software das Fundament des gesamten Projekts. Sie definiert die Systemlogik, steuert den Datenfluss, implementiert die KI-Modelle und stellt sicher, dass aus abstrakten Verkehrsdaten konkrete Optimierungsentscheidungen entstehen. Die nachfolgenden Kapitel widmen sich daher detailliert dem Aufbau, der Funktionsweise und den besonderen Herausforderungen der Programmierung von LanePilot.

Projekt: Gesamtarchitektur und Struktur

Um ein komplexes, KI-gestütztes System wie LanePilot zuverlässig entwickeln, testen und erweitern zu können, war eine klar definierte und durchdachte Softwarearchitektur unerlässlich. Bereits zu Beginn der Projektphase wurde deshalb großer Wert auf eine logisch getrennte und skalierbare Projektstruktur gelegt. Diese Architektur ermöglicht es, einzelne Komponenten unabhängig voneinander zu entwickeln, auszutauschen oder zu erweitern, ohne das Gesamtsystem zu destabilisieren. Die folgende Struktur dient nicht der technischen Detailtiefe, sondern der Verdeutlichung, dass das Projekt modular, reproduzierbar und erweiterbar aufgebaut wurde.

Das Projekt ist in mehrere Hauptverzeichnisse gegliedert, die jeweils eine klar abgegrenzte Aufgabe erfüllen. Im Root-Verzeichnis befinden sich globale Konfigurations- und Steuerdateien wie `project_config.yaml`, `requirements.txt`, `entrypoint.sh` sowie die Dokumentations- und Lizenzdateien. Diese Dateien definieren globale Parameter, Abhängigkeiten und Startpunkte des Systems und stellen sicher, dass das Projekt reproduzierbar und plattformübergreifend ausgeführt werden kann.

Der Ordner `ai/` bildet das Herzstück der Software. Hier sind sämtliche Module zur Konfiguration, zum Training und zur Evaluation der KI-Modelle angesiedelt. Dazu zählen sowohl die neuronalen Netze selbst als auch die Trainingspipelines, Loss-Funktionen, Optimizer, Learning-Rate-Scheduler und Exportmechanismen. Die klare Trennung innerhalb dieses Ordners erlaubt es, verschiedene KI-Ansätze wie Multi-Layer-Perceptrons, Graph Attention Networks oder Reinforcement-Learning-Modelle parallel zu testen und objektiv miteinander zu vergleichen.

Ergänzend dazu enthält `shared_src/` alle wiederverwendbaren Softwarekomponenten, die nicht direkt an eine spezifische KI oder Hardware gebunden sind. Dazu zählen Module zur Datenvorverarbeitung, Hilfsfunktionen, Netzwerkkommunikation sowie ein zentrales

Konfigurationssystem. Diese Wiederverwendbarkeit reduziert Redundanzen im Code und erhöht gleichzeitig die Wartbarkeit und Lesbarkeit des Projekts.

Für die Ausführung auf realer Hardware wurde der Ordner `firmware/` eingeführt. In diesem befinden sich gerätespezifische Laufzeitskripte für Systeme wie den NVIDIA Jetson oder den Raspberry Pi. Diese Skripte übernehmen Aufgaben wie die KI-Inferenz, die Bildverarbeitung zur Laufzeit sowie die Ansteuerung von Aktoren und Schnittstellen. Die Trennung zwischen Entwicklungs- und Laufzeitcode stellt sicher, dass die Firmware auf die begrenzten Ressourcen eingebetteter Systeme optimiert werden kann.

Abgerundet wird die Architektur durch zusätzliche Verzeichnisse wie `scripts/`, die Hilfsprogramme für uns als Entwickler enthalten, sowie `config/` und `opencv/`, welche Docker-spezifische Konfigurationen und Build-Prozesse kapseln. Vor allem der Einsatz von Docker⁸ ermöglichte eine konsistente Entwicklungs- und Laufzeitumgebung, unabhängig vom verwendeten Betriebssystem.

Insgesamt zeichnet sich die Softwarearchitektur von LanePilot durch eine klare Struktur, saubere Trennung von Zuständigkeiten und hohe Erweiterbarkeit aus. Diese Eigenschaften waren entscheidend, um ein technisch anspruchsvolles Projekt mit vielen Abhängigkeiten kontrolliert umzusetzen und bilden die Grundlage für die im nächsten Abschnitt beschriebene Daten- und Verarbeitungspipeline.

Datenerfassung, -annotation und -aufbereitung

Die Grundlage unserer Software bildet eine sorgfältig konzipierte Datenpipeline, die reale und synthetische Verkehrsdaten miteinander kombiniert. Ziel dieser Pipeline war es, reale Verkehrssituationen strukturiert zu erfassen, sie in eine maschinenlesbare Form zu überführen und gleichzeitig eine ausreichende Menge an Trainingsdaten mit genügend Varianz für das Training unterschiedlicher KI-Modelle bereitzustellen. Dabei wurde bewusst zwischen Trainings-, Simulations- und Validierungsdaten unterschieden, um die Aussagekraft der Ergebnisse zu erhöhen.

Für die Objekterkennung wurde ein YOLO-Modell (vgl. „[Darstellung der Projektidee](#)“) eingesetzt, das nicht auf realen Autobahnaufnahmen trainiert wurde, sondern auf eigens aufgenommenen Bildern des Modellaufbaus. Diese Bilder zeigen die physischen Modellfahrzeuge, Fahrspuren, Leitplanken und weitere relevante Elemente unter kontrollierten Bedingungen. Die Annotation dieser Bilder erfolgte mit Roboflow⁹ unter Verwendung von sog. „Bounding Boxes“ (dt. „Begrenzungsrechteck“). Bounding Boxes sind durch ihre vier Eckpunkte deutlich simpler und ressourcensparender als Polygonmasken mit mehreren Eckpunkten und für die einfache

⁸ **Docker:** Plattform, die es Entwicklern ermöglicht, Anwendungen zusammen mit all ihren Abhängigkeiten (Code, Laufzeitumgebung, Bibliotheken) in standardisierte, isolierte Pakete („Container“) zu verpacken, auszuführen und zu verwalten.

⁹ **Roboflow:** Eine Plattform, die den Entwicklungs-, Trainings- und Deployment-Prozess von Computer-Vision-Modellen vereinfacht.

Eingrenzung bzw. Rahmung der Fahrzeugposition völlig ausreichend (siehe [Abb. 10 „Fahrzeuwerkerkennung mittels YOLOv11“](#)).

Die realen Verkehrsaufnahmen von Autobahnabschnitten erfüllen im Projekt eine andere Funktion. Sie dienen nicht als Trainingsdatensatz für das YOLO-Modell, sondern als Validierungs- und Vergleichsdatensatz für die späteren Optimierungsergebnisse. Diese Aufnahmen zeigen reale, häufig ineffiziente Verkehrssituationen mit unausgeglichene Spurbelegungen und dienen dazu, die Entscheidungen des späteren Reinforcement-Learning-Modells auf reale Szenarien anzuwenden und qualitativ mit dem Ist-Zustand zu vergleichen.

Nach der Objekterkennung werden die vom YOLO-Modell gelieferten Polygon- bzw. Boxkoordinaten extrahiert und weiterverarbeitet. Jedes erkannte Fahrzeug wird als eigenständige Instanz betrachtet, deren Position und Ausdehnung im Bildraum bekannt ist. Durch geometrische Operationen, insbesondere die Überlappung von Fahrzeugpolygonen mit Spurpolygonen, kann jedem Fahrzeug zuverlässig eine Spur zugeordnet werden. Diese geometrische Zuordnung bildet die Grundlage für die spätere Szenarienbildung.

Um Verkehrssituationen als zusammenhängende Zustände modellieren zu können, werden die erkannten Fahrzeuge zu Graph-Szenarien zusammengefasst. Jedes Szenario repräsentiert einen Zeitpunkt beziehungsweise einen Frame und besteht aus mehreren Fahrzeugen, die gemeinsam betrachtet werden. In diesem Kontext werden Edge-Indizes generiert, welche die Beziehungen zwischen den Fahrzeugen beschreiben. Die Edges werden programmatisch auf Basis räumlicher und spurbezogener Kriterien erzeugt: Fahrzeuge auf derselben Spur sowie auf benachbarten Spuren werden miteinander verbunden, da sie potenziell Einfluss auf gegenseitige Entscheidungen haben. Fahrzeuge ohne relevante räumliche Nähe oder Interaktion werden bewusst nicht verbunden, um den Graphen übersichtlich und informationsdicht zu halten (siehe [Abb. 11 „Beispielhafte graphische Darstellung eines Verkehrsszenarios“](#)).

Neben der reinen Struktur des Graphen spielt die Präzision der numerischen Features eine bedeutende Rolle. Aus den erkannten Objekten werden Feature-Vektoren extrahiert, die unter anderem spurbezogene Informationen, dynamische Größen wie Geschwindigkeit und Beschleunigung sowie globale Kontextinformationen wie die aktuelle Spurenauslastung enthalten. Da diese Features unterschiedliche Wertebereiche besitzen, wird eine Z-Score-Normalisierung über den gesamten Datensatz angewendet. Dadurch werden alle Features vergleichbar skaliert, was die Stabilität des Trainings erhöht und die Modelle dabei unterstützt, schneller zu konvergieren und zuverlässiger ein Optimum zu erreichen.

Um die Vielfalt des Datensatzes zusätzlich zu erhöhen, wurde eine künstliche Datenerweiterung implementiert. Dabei werden synthetische Verkehrsszenarien generiert, indem Fahrzeugpolygone mit realistischer Größe erzeugt und zufällig auf verschiedenen Spuren platziert werden. Geschwindigkeit, Beschleunigung und Fahrzeuganzahl werden innerhalb physikalisch sinnvoller Grenzen zufällig variiert. Diese synthetischen Szenarien werden identisch zu realen Szenarien

verarbeitet und erweitern den Datensatz um ein Vielfaches, ohne die strukturelle Konsistenz zu verlieren.

Die final aufbereiteten Daten werden als PyTorch ¹⁰ (.pt) -Dateien gespeichert. Jede Datei repräsentiert ein vollständiges Verkehrsszenario und enthält die Feature-Vektoren aller Fahrzeuge, die zugehörigen Zielwerte sowie die berechneten Edge-Indizes. Diese Form der Speicherung ermöglicht ein effizientes Laden während des Trainings und erlaubt es, unterschiedliche Modellarchitekturen flexibel auf denselben Datensatz anzuwenden.

Modellarchitektur und Trainingspipeline

Die Entwicklung der künstlichen Intelligenz in LanePilot verlief nicht linear, sondern iterativ und problemgetrieben. Im Laufe des Projekts kamen drei unterschiedliche KI-Ansätze zum Einsatz, die jeweils auf den Erkenntnissen und Grenzen der vorherigen Architektur aufbauten. Dieser Wandel war notwendig, um der zunehmenden Komplexität des Problems gerecht zu werden und gleichzeitig realistische, skalierbare Lösungen zu entwickeln.

Zu Beginn wurde ein klassisches Feed-Forward-Neural-Network (MLP) eingesetzt. Dieses Modell erhielt für jedes Fahrzeug einen Feature-Vektor und sollte unabhängig von anderen Fahrzeugen die optimale Zielspur vorhersagen. Der Vorteil dieses Ansatzes lag in seiner Einfachheit: Das Training war stabil, die Implementierung überschaubar und erste brauchbare Ergebnisse konnten schnell erzielt werden. Insbesondere für isolierte Entscheidungen, bei denen ausschließlich individuelle Eigenschaften wie Geschwindigkeit, aktuelle Spur und Spurenauslastung dominieren, zeigte das MLP solide Genauigkeiten von ~71,5%.

Allerdings offenbarte sich relativ früh ein grundlegendes Problem dieses Ansatzes: Verkehr ist kein unabhängiges Regressionsproblem. Fahrzeuge beeinflussen sich gegenseitig, insbesondere auf derselben oder benachbarten Spur. Das MLP betrachtete jedes Fahrzeug isoliert und konnte diese Wechselwirkungen nur indirekt über gemeinsame Features wie die Spurenauslastung abbilden. Lokale Effekte, etwa Staus durch einzelne langsame Fahrzeuge oder das Zusammenspiel mehrerer schneller Fahrzeuge, ließen sich mit diesem Modell nur unzureichend erfassen. Diese Einschränkung führte zur Suche nach einer Architektur, die explizit Relationen zwischen Fahrzeugen modellieren kann.

Als nächster Schritt wurde ein Graph Attention Network (GATv2) eingeführt. In diesem Ansatz wird jedes Verkehrsszenario als Graph interpretiert, in dem Fahrzeuge als Knoten („Nodes“) und ihre Beziehungen als Verbindungsvektoren („Edges“) dargestellt sind. Die im Code implementierte `VehicleState`-Klasse spielte hierbei eine wichtige Rolle: Sie fungierte als Abstraktion eines Fahrzeugs über mehrere Frames hinweg, fasste dynamische Zustände wie Geschwindigkeit und Beschleunigung zusammen und stellte den Feature-Vektor für den Graph bereit. Zusätzlich

¹⁰ **PyTorch**: Eine Open-Source-Bibliothek für maschinelles Lernen und Deep Learning. Unsere KI-Modelle wurden damit implementiert und verwaltet.

lieferte sie die notwendigen Informationen, um Edge-Indizes auf Basis von Spurzugehörigkeit und räumlicher Nähe zu berechnen.

Der Einsatz eines GATv2-Modells ermöglichte es erstmals, lokale Interaktionen explizit zu berücksichtigen. Durch den Attention-Mechanismus konnte das Modell lernen, welche Nachbarfahrzeuge für die Entscheidung eines bestimmten Fahrzeugs relevant sind. Fahrzeuge auf derselben Spur oder direkt benachbarten Spuren erhielten dabei ein höheres Gewicht als entfernte Fahrzeuge. Theoretisch passte dieser Ansatz sehr gut zur Problemstellung, da Verkehrsflüsse stark durch lokale Beeinflussungen geprägt sind.

In der Praxis zeigte sich jedoch, dass dieser Ansatz mit erheblichen Herausforderungen verbunden war. Die Generierung sinnvoller Edge-Indizes erwies sich als schwierig und fehleranfällig, insbesondere bei wechselnder Fahrzeuganzahl und unvollständigen oder komplexeren Szenarien. Kleine Änderungen in der Edge-Logik führten teilweise zu starken Leistungseinbrüchen. Zudem war das Modell deutlich instabiler als das MLP und neigte trotz umfangreicher Hyperparameter-Optimierung zu schwankender Trainingsdynamik. Obwohl Spitzen-Genauigkeiten von ~79,25% erreicht wurden, blieb der tatsächliche Mehrwert gegenüber dem deutlich einfacheren MLP begrenzt. Dies führte zu der Erkenntnis, dass das Problem möglicherweise grundlegend anders angegangen werden muss.

Der entscheidende Schritt erfolgte mit dem Übergang zu Reinforcement Learning (RL). Anstatt das Problem weiterhin als überwachte Regressionsaufgabe zu betrachten, wurde es nun als sequenzielles Entscheidungsproblem modelliert. In realem Verkehr werden Entscheidungen nicht isoliert getroffen, sondern beeinflussen zukünftige Zustände des Systems. Genau diese zeitliche Abhängigkeit konnte weder das MLP noch das GAT vollständig abbilden.

Mit Hilfe von Gymnasium¹¹ wurde eine simulierte Verkehrsumgebung aufgebaut, in der Fahrzeuge als Agenten agieren. Ein Agent trifft Entscheidungen wie Spurwechsel oder Verbleib auf der aktuellen Spur und erhält dafür eine Belohnung, die den Verkehrsfluss, die Durchschnittsgeschwindigkeit und die Vermeidung von Überlastung berücksichtigt. In diesem Ansatz entfällt die explizite `VehicleState`-Logik weitgehend, da der Zustand des Systems direkt durch die Umgebung beschrieben wird. Das Modell lernt nicht mehr aus vorgegebenen Labels, sondern aus den Konsequenzen seiner eigenen Entscheidungen.

Dieser Wechsel zu Reinforcement Learning stellte einen wichtigen und unvermeidbaren Schritt in Richtung Realitätstreue dar. Während MLP und GAT lediglich bestehende Verkehrszustände bewerten konnten, ist das RL-Modell in der Lage, aktiv Optimierungsstrategien zu entwickeln. Die zuvor aufgenommenen realen Verkehrsvideos dienen hierbei als Referenz, um die vom RL-Modell erzeugten optimierten Verkehrsszenarien mit realen, ineffizienten Situationen zu vergleichen.

¹¹ **Gymnasium**: Ein Open-Source-Toolkit zur Entwicklung und Bewertung von Reinforcement-Learning-(RL)-Agenten. Hinweis: Der Begriff hat keinen Zusammenhang zur Schulform.

Zusammenfassend spiegelt der Wandel der KI-Architekturen den Erkenntnisgewinn während des Projekts wider: von einfachen, lokalen Entscheidungen hin zu relationalen Modellen und schließlich zu einem ganzheitlichen, dynamischen Optimierungsansatz. Dieser iterative Prozess verdeutlicht, dass technische Perfektion nicht nur im Endergebnis liegt, sondern vor allem durch den Weg dorthin erreicht wird.

Trainingspipeline & Optimierungsstrategien

Nach der Festlegung der jeweiligen KI-Architektur bildete die Trainingspipeline das zentrale Bindeglied zwischen Datensatz und Modellleistung. Ziel war es, einen reproduzierbaren und möglichst stabilen Trainingsprozess zu schaffen, der sowohl für klassische neuronale Netze als auch für Graph-basierte Modelle geeignet ist.

Der Trainingsprozess beginnt mit dem Laden der konfigurierten Modellparameter (siehe [Abb. 12 „Graph Attention Network – Modell-Parameter“](#)) sowie der Aufteilung des Datensatzes in Trainings-, Validierungs- und Test-Splits. Diese Trennung war besonders wichtig, da die Datensätze teilweise synthetisch erzeugt wurden und somit eine strikte Trennung zwischen Trainings- und Evaluierungsdaten notwendig war, um Overfitting zu vermeiden. Die Datensätze werden anschließend über PyTorch-Dataloader mit einer vergleichsweise hohen Batchgröße von 512 geladen, um die GPU-Auslastung zu optimieren.

Da die Klassenverteilung der Zielspuren stark unausgeglichene war, insbesondere zugunsten der äußeren Spuren, wurde eine dynamische Klassengewichtung implementiert. Diese basiert auf der absoluten Häufigkeit der jeweiligen Spurklasse im Trainingsdatensatz und wird direkt an die Cross-Entropy-Loss-Funktion übergeben. Dadurch wird verhindert, dass das Modell pauschal Entscheidungen trifft, bei denen hauptsächlich die im Datensatz am häufigsten vorkommenden Klassen vorhergesagt werden.

Als Optimierungsverfahren kommt AdamW¹² zum Einsatz, da dieser im Vergleich zum klassischen Adam-Optimizer eine entkoppelte Gewichtsnormierung (Weight Decay) verwendet und sich in Experimenten als deutlich stabiler erwiesen hat. Die Lernrate wird nicht konstant gehalten, sondern über einen CosineAnnealingWarmRestarts-Scheduler¹³ dynamisch angepasst. Dieser Ansatz hilft dem Modell, sich regelmäßig aus lokalen Minima zu befreien und trägt so zu einer besseren Generalisierung bei, wenn die Lernkurve zu stagnieren droht.

Um unnötig lange Trainingszeiten zu vermeiden, wurde ein Early-Stopping-Mechanismus implementiert, der das Training beendet, sobald sich der Validierungsverlust über mehrere Epochen hinweg nicht mehr verbessert. Parallel dazu werden Modell-Checkpoints gespeichert, sodass jederzeit auf das leistungsstärkste Modell zurückgegriffen werden kann. Am Ende des Trainingsprozesses wird das finale

¹² **AdamW**: Ein Optimierer der häufig bei Computer-Vision-Aufgaben eingesetzt wird. Seine Fähigkeit, Überanpassungen zu verhindern, macht ihn ideal für Aufgaben mit großen Datensätzen und komplexen Architekturen.

¹³ **CosineAnnealingWarmRestarts-Scheduler**: Eine Lernratenstrategie, bei der die Lernrate periodisch einer Kosinusfunktion folgt und nach jedem Zyklus zurückgesetzt wird.

Modell sowohl im nativen PyTorch-Format als auch im ONNX¹⁴-Format exportiert, um eine spätere Hardwarebeschleunigung zu ermöglichen.

Doch all diese konkreten Hyperparameter sind weniger entscheidend als das zugrunde liegende Prinzip: Stabilität, Generalisierung und Reproduzierbarkeit.

Inferenz, Deployment & Laufzeitsystem

Ein elementares Ziel von LanePilot war es von Beginn an, die entwickelten KI-Modelle nicht nur offline zu trainieren, sondern sie auch in einem realitätsnahen Laufzeitsystem einzusetzen. Daher wurde großer Wert auf eine saubere Trennung zwischen Trainings- und Inferenzcode gelegt.

Die Inferenzpipeline ist containerbasiert umgesetzt und läuft vollständig in Docker-Containern, sowohl auf dem NVIDIA Jetson als auch auf dem Raspberry Pi. Dadurch wird sichergestellt, dass Abhängigkeiten, CUDA¹⁵-Versionen und Bibliotheken reproduzierbar und unabhängig vom Host-System sind. Beide Docker-Firmware-Images sind von uns kompiliert und öffentlich aufrufbar. Das trainierte Modell wird innerhalb des Containers entweder direkt als PyTorch-Modell geladen oder über ONNX für performantere Ausführungen vorbereitet. Eine TensorRT¹⁶-Integration wurde bereits konzeptionell umgesetzt, um zukünftig eine noch geringere Latenz zu erreichen.

Während der Inferenz wird das aktuelle Fahrzeugszenario frameweise verarbeitet. Die Feature-Extraktion erfolgt dabei genauso wie in der Trainingsphase, um sogenannte „Train-Inference-Mismatches“ zu vermeiden, bei denen falsche Ergebnisse aufgrund unterschiedlicher Preprocessing-Methoden auftreten könnten. Der Edge-Index für Graph-basierte Modelle wird nur dann neu berechnet, wenn sich die Anzahl der Fahrzeuge oder deren IDs ändern, was die Laufzeit erheblich reduziert. Dadurch bleibt das System auch bei höheren Fahrzeugzahlen echtzeitfähig.

Die gesamte Inferenzlogik ist so gestaltet, dass sie unabhängig von der zugrunde liegenden Hardware funktioniert. Dies erlaubt es, das System sowohl in einem Demonstrationsaufbau als auch in einer zukünftigen, realen Verkehrsinfrastruktur einzusetzen. Die Container können automatisiert gestartet, überwacht und aktualisiert werden, was insbesondere für verteilte Systeme von großer Bedeutung ist.

Besondere Herausforderungen & Learnings

Während der Projektentwicklung traten zahlreiche technische Herausforderungen auf, die maßgeblich zur Weiterentwicklung des Systems beigetragen haben. Eine der größten Schwierigkeiten bestand in der Generierung stabiler Edge-Indizes für das Graph Attention Network. Kleine Änderungen in der Fahrzeuganzahl oder deren Positionen führten zu stark variierenden Graphstrukturen, was das Training

¹⁴ **ONNX**: Ein quelloffenes Format zur Repräsentation von Deep-Learning-Modellen. Mit ONNX können KI-Entwickler Modelle zwischen verschiedenen Tools austauschen und die für sie beste Kombination dieser Tools wählen.

¹⁵ **CUDA**: Eine NVIDIA-Technologie zur GPU-Beschleunigung von Rechenoperationen. KIs können damit effizienter auf leistungsstarken Grafikkarten trainiert werden.

¹⁶ **TensorRT**: Eine Optimierungs- und Laufzeitumgebung für Deep-Learning-Inferenz, die die Ausführung neuronaler Netzwerke auf NVIDIA-GPUs durch geringere Latenz und höheren Durchsatz deutlich effizienter macht.

instabil machte und letztlich einer der Gründe für den späteren Architekturwechsel war. Da Edge-Indizes programmatisch erzeugt wurden, war es auch hier schwierig, passende Regeln zu definieren, wann eine Beziehung zwischen zwei Fahrzeugen berücksichtigt oder vernachlässigt werden sollte.

Auch die Datenqualität stellte eine erhebliche Herausforderung dar. Beim Skalieren des Datensatzes kam es häufig zu ungültigen Polygonformen oder Fahrzeugen, die keiner Spur eindeutig zugeordnet werden konnten. Diese Fälle mussten algorithmisch erkannt und verworfen werden, um fehlerhafte Trainingsbeispiele zu vermeiden. Zudem war die Klassenverteilung stark unausgeglich, was ohne gezielte Gewichtung zu einer deutlichen Verzerrung der Modellvorhersagen führte. So befanden sich in den Trainingsdaten beispielsweise deutlich mehr Fahrzeuge auf der linken Spur, was zu einem „Bias“ bzw. einer Voreinstellung führte. Die KI neigte daher dazu, Fahrzeuge eher auf der linken Spur zu platzieren.

Ein weiteres Problem betraf die Softwareinfrastruktur bei den Firmware-Images. Aufgrund von Lizenzbeschränkungen seitens NVIDIA durfte das Video Codec SDK¹⁷ nicht öffentlich verteilt werden, weshalb ein manueller OpenCV¹⁸-Build mit CUDA-Unterstützung notwendig war. Die SDK war ein essenzieller Treiber für die Kommunikation mit der Kamera und musste daher zwingend implementiert werden. Zusätzlich war eine lokale Kompilierung der Docker-Images auf Windows-Systemen nicht möglich, sodass ein automatisierter ARM64-Build über GitHub Actions und QEMU¹⁹ implementiert wurde.

Auch das Docker-Networking erwies sich als komplex: Die Ethernet-Verbindung zwischen Raspberry Pi und Jetson Orin Nano konnte erst durch die Konfiguration physischer und virtueller Netzwerkinterfaces innerhalb der Container sowie die Vergabe ausreichender Zugriffsrechte ermöglicht werden, was umfangreiche Tests und Iterationen erforderte. Diese Erfahrungen verdeutlichten, dass ein funktionierendes KI-System immer auch eine robuste Infrastruktur benötigt.

Soziale Auswirkungen & Innovation

a) Direkte Auswirkungen auf Menschen in unserer Umgebung

In urbanen Räumen und dicht besiedelten Gebieten werden die Folgen ineffizienter Verkehrssysteme täglich sichtbar. Verkehrsstaus, erhöhte Lärmbelastung, Zeitverluste und negative Umweltauswirkungen prägen den Alltag vieler Menschen. Die entwickelte Lösung setzt genau an diesen Problempunkten an, indem Fahrzeuge gezielt und intelligent auf verfügbare Fahrspuren verteilt werden, um den Verkehrsfluss nachhaltig zu verbessern.

¹⁷ **Video Codec SDK:** Ein Toolkit von NVIDIA, welches Zugriff auf die hardwarebeschleunigte Video-Kodierung (Encoding) und -Dekodierung (Decoding) auf Grafikkarten gibt, um Videos effizient zu komprimieren und zu dekomprimieren.

¹⁸ **OpenCV:** Eine kostenlose, quelloffene Softwarebibliothek mit Tausenden von Algorithmen für Computer Vision, Bildverarbeitung und maschinelles Lernen.

¹⁹ **QEMU:** Ein Emulator und Virtualisierer, der es ermöglicht, virtuelle Maschinen (VMs) zu erstellen und verschiedene Betriebssysteme auf einem Host-System auszuführen, indem es Hardware wie CPUs, Speicher und Peripheriegeräte nachbildet.

Der Einsatz von LanePilot kann im lokalen Umfeld mehrere positive Effekte entfalten:

- Zeitgewinn für Pendler: Weniger Staus bedeuten weniger verlorene Lebenszeit. Berufspendler, Handwerker, Pflegekräfte oder Familien auf dem Schulweg profitieren direkt.
- Geringerer Stress im Straßenverkehr: Gleichmäßigere Auslastung der Spuren reduziert das aggressive Fahrverhalten, Drängeln oder unnötige Spurwechsel und somit Unfälle.
- Lärminderung in Wohngebieten: Fließender Verkehr erzeugt weniger Lärm als ständiges Bremsen und Anfahren → ein Gewinn für die Lebensqualität in der Stadt.
- Förderung nachhaltiger Mobilität: Durch effizientere Verkehrsverteilung sinkt der durchschnittliche Kraftstoffverbrauch, was auch im privaten Bereich Kosten spart.

Langfristig kann LanePilot auch Kommunen helfen, den Bedarf an neuen Straßen besser zu planen oder sogar ganz zu vermeiden. Das Projekt kann auch als pädagogisches Beispiel dienen, um Technikbegeisterung bei Jugendlichen zu fördern (z. B. an Schulen, MINT-Initiativen).

b) Potenzielle Auswirkungen auf die globale Bevölkerung

Auch weltweit ist Verkehr eines der maßgebenden Probleme des 21. Jahrhunderts. In Millionenmetropolen wie Jakarta, Lagos, São Paulo oder New Delhi sind tägliche Mega-Staus Alltag. Verkehrsstaus verursachen dort massive wirtschaftliche Schäden, beeinträchtigen die Gesundheit und gefährden die Sicherheit.

LanePilot hat das Potenzial, diese Herausforderungen global positiv zu beeinflussen:

- Reduktion von CO₂-Emissionen: Jedes Fahrzeug, das weniger im Stau steht, verbrennt weniger Kraftstoff. Weltweit summiert sich das auf Millionen Tonnen CO₂ jährlich.
- Höhere Sicherheit im Straßenverkehr: Durch die Vermeidung von plötzlichen Spurwechseln und stockendem Verkehr sinkt das Unfallrisiko. Besonders in Ländern mit schwacher Verkehrsinfrastruktur kann das Leben retten.
- Skalierbarkeit: Das System kann auf unterschiedlichste Straßeninfrastrukturen angepasst werden, ob in hochentwickelten Ländern mit V2I-Technologie oder in Entwicklungsregionen mit einfachen Anzeigetafeln.
- LanePilot lässt sich konzeptionell nahtlos in bestehende Smart-City-Ansätze integrieren. Es ergänzt andere Systeme wie intelligente Ampeln, vernetzte Fahrzeuge und Verkehrsprognosemodelle.

c) Gesellschaftlicher Nutzen durch Technologieakzeptanz

Ein weiterer sozialer Aspekt ist die Akzeptanz und der verantwortungsvolle Einsatz von KI-Systemen. In vielen gesellschaftlichen Diskussionen wird Künstliche Intelligenz als unkontrollierbare, abstrakte Bedrohung wahrgenommen.

LanePilot zeigt, dass KI verständlich, nachvollziehbar und sinnvoll eingesetzt werden kann. Sie dient nicht als Ersatz des Menschen, sondern als Werkzeug zur Verbesserung bestehender Systeme.

Beispiele für positive Effekte sind:

- Vertrauen in Technologie: Durch sichtbare Erfolge im Alltag (z. B. weniger Staus) wächst das Vertrauen in digitale Lösungen
- Barrierefreiheit im Verkehr: Gleichmäßiger Verkehrsfluss hilft auch älteren Menschen oder Menschen mit Mobilitätseinschränkungen, sich sicherer zu bewegen.
- Demonstration von Machbarkeit: LanePilot ist nicht nur ein Konzept, sondern ein funktionierendes Modell. Es macht Zukunftstechnologie greifbar.

d) Mögliche Risiken und ethische Fragen

Natürlich bringt jede neue Technologie auch Herausforderungen mit sich. Wir haben diese Aspekte frühzeitig reflektiert:

Risiken:

- Fehlinterpretationen durch KI: Wenn Bilddaten falsch analysiert werden, könnten Fahrzeuge auf ungeeignete Spuren gelenkt werden. Das wäre insbesondere im realen Verkehr kritisch.
- Ungleichheit bei der Umsetzung: Länder mit schlechter Infrastruktur könnten von solchen Systemen ausgeschlossen bleiben, während andere sie weiterentwickeln.
- Datenschutz bei V2I-Kommunikation: Die Erhebung von Fahrzeugdaten muss sicher und anonymisiert erfolgen, um keine Persönlichkeitsrechte zu verletzen.

Unser Umgang damit:

- Wir setzen auf Transparenz und DSGVO-Konformität, erklären alle Systementscheidungen nachvollziehbar.
- Unser Modell ist modular, d. h. es kann auch ohne Internetanbindung oder sensiblen Daten funktionieren.
- Bei echter Anwendung muss die Verantwortung immer beim Menschen bleiben. Die KI liefert Empfehlungen, keine zwingenden Anweisungen.

e) Innovationsgrad und gesellschaftlicher Fortschritt

LanePilot stellt eine technologische und gesellschaftliche Innovation dar:

- Technologisch, weil wir ein komplexes Zusammenspiel aus Objekterkennung, graph- bzw. RL-basierter Logik und physischer Spursteuering umgesetzt haben.
- Gesellschaftlich, weil wir zeigen, dass auch junge Teams mit begrenzten Ressourcen reale Probleme kreativ und lösungsorientiert angehen können.
- Pädagogisch, weil unser Projekt eine hervorragende Möglichkeit bietet, um technische Bildung und gesellschaftliches Bewusstsein zu verbinden.

f) Ausblick: Ein Schritt Richtung Zukunft

LanePilot ist mehr als ein Robotikmodell: Es ist ein Konzept mit Vision. Es zeigt, dass ein intelligenter Umgang mit Technik Verkehrsprobleme nicht nur lindern, sondern grundlegend neu denken kann.

In einer Zeit, in der Klimawandel, Urbanisierung und Digitalisierung zusammenkommen, sind Projekte wie LanePilot nicht nur technisch spannend, sondern auch gesellschaftlich notwendig.

Weitere nicht-wissenschaftliche Details (Programmcode, Bilder, Making of):



GitHub: AppSolves



Instagram: @appsolves.dev

Quellcode: <https://github.com/AppSolves/LanePilot>

Literaturverzeichnis

- ZEIT Online: „ADAC-Staubilanz Autobahnen – Verkehr“, Presseartikel, Zugriff am 08.10.2025. <https://www.zeit.de/mobilitaet/2025-02/adac-staubilanz-autobahnen-verkehr>
- Centralbestellung: Verkehrsleitsysteme und Baustellensicherung, Händlerwebsite, Zugriff am 02.11.2025. <https://www.centralbestellung.de/Baustellensicherung/Verkehrsleitsysteme/>
- EBM Verkehrs- und Parkleitsysteme: Herstellerinformation, Zugriff am 02.11.2025. <https://ebm-os.de/park-und-verkehrsleitsystem/>
- StackFuel: Grundlagen der Robotik, Bildungsplattform / Fachartikel, Zugriff am 02.11.2025. <https://stackfuel.com/de/blog/grundlagen-der-robotik-definition-arten-und-einsatz/>
- HortiTrade: Förderband (Produktseite), Händlerwebsite, Zugriff am 04.11.2025. <https://www.hortitrade.com/de/katalog/mb0842048/forderband-potveer-457-x-50-cm-1>
- Dorner Conveyors: Fördertechnik-Lösungen, Herstellerinformation, Zugriff am 04.11.2025. <https://www.dornerconveyors.com/europe/de/loesungen/zusammenfuehrungs-umlenk-und-sortierfoerderer>
- StackOverflow: Entwicklerdiskussionen zu technischen Fragestellungen, Online-Entwicklerforum, Zugriff am 05.11.2025. <https://stackoverflow.com/questions>
- IWB GmbH: Förderbandtechnik, Herstellerinformation, Zugriff am 08.11.2025. <https://iwb.gmbh/foerderbaender/>
- Matco International: Förderbandlösungen, Herstellerinformation, Zugriff am 08.11.2025. <https://matco-international.com/de/forderband/>
- Wikipedia: „Weiche (Verkehrstechnik)“, Online-Lexikon, Zugriff am 08.11.2025. [https://de.wikipedia.org/wiki/Weiche_\(Verkehrstechnik\)](https://de.wikipedia.org/wiki/Weiche_(Verkehrstechnik))
- Wikipedia: „Schiebebühne“, Online-Lexikon, Zugriff am 09.11.2025. <https://de.wikipedia.org/wiki/Schiebeb%C3%BChne>
- Carrera4Fun: iTrack-System, Hobby- und Technikportal, Zugriff am 09.11.2025. https://www.carrera4fun.de/8_itrack/i-prox.htm
- YouTube: Demonstrationsvideo zu Fördertechnik, Video-Plattform, Zugriff am 09.11.2025. <https://www.youtube.com/watch?v=tcaak6fuK8I>
- Pollin Electronic: Elektronik-Versandhandel, Händlerwebsite, Zugriff am 09.11.2025. <https://www.pollin.de/>
- Reichelt Elektronik: Elektronik-Versandhandel, Händlerwebsite, Zugriff am 09.11.2025. <https://www.reichelt.de/>
- Voelkner: Elektronik-Fachhandel, Händlerwebsite, Zugriff am 09.11.2025. <https://www.voelkner.de/>
- Medium: Online-Publishing-Plattform für Fachartikel, Blogplattform, Zugriff am 13.12.2025. <https://medium.com/>
- Fraunhofer-Gesellschaft: KI und Robotik – Forschungsübersicht, Forschungsinstitut, Zugriff am 22.11.2025. <https://www.fraunhofer.de/de/forschung/aktuelles-aus-der-forschung/ki-robotik.html>

- Automationspraxis: KI als Chance für die Robotik, Fachzeitschrift, Zugriff am 22.11.2025.
<https://automationspraxis.industrie.de/ki/ki-eine-grosse-chance-fuer-die-robotik/>
- Elektronikforschung.de: Robotik und Automatisierung, Fachportal, Zugriff am 22.11.2025.
<https://www.elektronikforschung.de/robotik>
- Bundesregierung: Genehmigungsbeschleunigung, Regierungsportal, Zugriff am 23.11.2025.
<https://www.bundesregierung.de/breg-de/service/archiv/genuehmigungsbeschleunigung-2187452>
- Bundesministerium für Verkehr: Planungsbeschleunigung Infrastruktur, Ministeriumswebsite, Zugriff am 23.11.2025. <https://www.bmv.de/DE/Themen/Mobilitaet/Infrastrukturplanung-Investitionen/Planungsbeschleunigung/planungsbeschleunigung.html>
- Deutscher Bundestag: Verkehrsnetz – Parlamentsdokumentation, Parlamentarisches Archiv, Zugriff am 23.11.2025. <https://www.bundestag.de/dokumente/textarchiv/2023/kw42-de-verkehrsnetz-971384>
- C-Roads Germany: C-ITS-Pilot Hamburg, Zugriff am 11.11.2025. <https://www.c-roads-germany.de/deutsch/c-its-piloten/c-its-pilot-hamburg/>
- sevDesk: Erfolgreiche Startups in Deutschland, Unternehmensblog, Zugriff am 12.12.2025.
<https://sevdesk.de/blog/die-15-erfolgreichsten-startups-deutschlands/>
- Deutsche Bank: UnternehmerTUM – Innovationsökosystem, Unternehmensinformation, Zugriff am 12.12.2025. https://www.db.com/what-we-do/focus-topics/Europe/UnternehmerTUM?language_id=1
- UnternehmerTUM: Innovations- und Gründerzentrum, Organisationswebsite, Zugriff am 12.12.2025. <https://www.unternehmertum.de/>
- Fraunhofer SAFE Intelligence: Sicheres Reinforcement Learning, Forschungsartikel, Zugriff am 17.12.2025. <https://safe-intelligence.fraunhofer.de/artikel/sicheres-reinforcement-learning>
- IBM: Reinforcement Learning – Grundlagen, Unternehmens-Fachartikel, Zugriff am 17.12.2025. <https://www.ibm.com/de-de/think/topics/reinforcement-learning>
- PyTorch: Reinforcement Q-Learning Tutorial, Entwicklerdokumentation, Zugriff am 27.11.2025. https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning.html
- ADAC: Stauprognose und Verkehrsinformationen, Verband / Interessenvertretung, Zugriff am 08.10.2025. <https://www.adac.de/verkehr/verkehrsinformationen/stauprognose/>
- Stern Online: „Staus in Deutschland nehmen zu – aus zwei guten Gründen“, Presseartikel, Zugriff am 09.10.2025. <https://www.stern.de/panorama/staus-in-deutschland-nehmen-zu---aus-zwei-guten-gruenden-35443924.html>
- DER SPIEGEL: Themenseite Verkehrsstau, Presseartikel / Dossier, Zugriff am 09.10.2025. <https://www.spiegel.de/thema/verkehrsstau/>
- GitHub: Code-Hosting-Plattform für Softwareprojekte, Entwicklerplattform, Zugriff am 18.12.2025. <https://github.com/>

- BZ Berlin: „Verkehr an Himmelfahrt in und um Berlin – Staugefahr“, Presseartikel, Zugriff am 09.10.2025. <https://www.bz-berlin.de/berlin/verkehr-an-himmelfahrt-in-und-um-berlin-stau-gefahr>
- Bundesministerium für Digitales und Verkehr: Smart-City-Navigator – Verkehrsprojekte, Regierungsportal, Zugriff am 15.10.2025. <https://www.de.digital/DIGITAL/Redaktion/DE/Smart-City-Navigator/Projekte/smarter-verkehrsleitsystem-zur-entlastung-derinnenstadt.html>
- Umweltbundesamt: Emissionen des Verkehrs, Bundesbehörde / Fachportal, Zugriff am 16.10.2025. <https://www.umweltbundesamt.de/daten/verkehr/emissionen-des-verkehrs>
- HORNBACH: Baumarkt und Materialbeschaffung, Händlerwebsite, Zugriff am 17.10.2025. <https://www.hornbach.de/>
- Strategyzer: The Business Model Canvas, Zugriff am 17.09.2025. <https://www.strategyzer.com/library/the-business-model-canvas>
- Guilin GLsun Science and Tech Group Co.,LTD.: Simplex vs. Half-Duplex vs. Duplex, Zugriff am 18.12.2025. <https://www.glsunmall.com/fiber-optic-articles/simplex-vs-half-duplex-vs-duplex.html>

Abkürzungsverzeichnis

B

Bd SI-Einheit [Baud]

C

CO₂..... Kohlenstoffdioxid

D

DSGVO *Datenschutz-Grundverordnung*

G

GATv2..... Graph Attention Network Version 2

GND Ground

K

KI Künstliche Intelligenz

M

MLP Multi-Layer Perceptron

O

ONNX Open Neural Network Exchange

Q

QEMU..... Quick Emulator

R

RL Reinforcement-Learning

RX *Receive*

T

TXTransmit

U

UART..... Universal Asynchronous Receiver / Transmitter

V

v. l. n. r. *von links nach rechts*

V2I.....Vehicle-To-Infrastructure

V2X.....Vehicle-To-Everything

W

WLAN.....*Wireless Local Area Network*

Y

YOLO You Only Look Once

Innovations- und Unternehmensaspekte

Der folgende Business Model Canvas inklusive der dargestellten Aspekte dient nicht der wirtschaftlichen Bewertung, sondern der Einordnung des Projekts als realitätsnahes Innovationskonzept. Es soll aufgezeigt werden, wie eine mögliche Umsetzung oder Kommerzialisierung in der Zukunft aussehen könnte.

The Business Model Canvas

Designed by:
LanePilot

Designed by:
K. Gönlüldinc + F. Helber

Date:
22.05.2025

Version:
1

Key Partners • Forschungseinrichtungen (z. B. für Verkehrssimulation, KI-Validierung) • Stadtverwaltungen & Behörden für reale Tests • Technologieleistern für Sensorik, Kamera- und Netzwerktechnik • Investoren & Fördergeber (z. B. mFUND, SmartCity-Programme) • Start-up-Inkubatoren & Acceleratoren für Marktstrategie und Skalierung	Key Activities • Weiterentwicklung und Training der KI-Modelle • Integration in reale Verkehrsnetze (Pilotprojekte) • Hardwareentwicklung & Tests für modulare Kamera-/Weicheneinheiten • Datenanalyse & Feedback-Auswertung zur Optimierung der Algorithmen • Akquise von Partnern & Finanzierung	Value Propositions • Reduktion von Staus durch KI-basierte Spurverteilung in Echtzeit • Effizientere Nutzung vorhandener Infrastruktur, ohne neue Straßen bauen zu müssen • Verbesserung der Luftqualität durch weniger Stop-and-Go • Steigerung der Verkehrssicherheit durch proaktive Spurwechselvorschläge • Modular einsetzbares System, skalierbar von Pilotprojekten bis zu ganzen Städten • V2I-fähig: bereit für Integration in autonome Fahrzeuge und Smart-City-Infrastrukturen	Customer Relationship • Langfristige Partnerschaften mit Kommunen (z. B. im Rahmen mehrjähriger Smart-City-Initiativen) • Support- und Wartungsservices für eingesetzte Hardware • Beratung & Datenauswertung zur Verkehrsoptimierung • Gemeinsame Entwicklung von Zusatzfunktionen mit Automobilherstellern Channels • Direkte Pilotprojekte mit Städten (kommunale Förderprogramme) • Kooperationen mit Technologiepartnern (z. B. Anbieter für Verkehrstechnik, Cloudlösungen) • Teilnahme an Messen und Wettbewerben für Smart Mobility und urbane Innovation • Online-Präsentation, z. B. über Website, Social Media, Whitepapers für Fachpublikum • Wissenschaftliche Publikationen in Bereichen Verkehr, KI, Automatisierung	Customer Segments • Städte & Kommunen, die unter Verkehrsproblemen leiden und an Smart-City-Lösungen interessiert sind • Verkehrsbehörden & Infrastrukturbetreiber, die Verkehrssicherheit und -effizienz steigern wollen • Automobilhersteller & Zulieferer, die V2I-Kommunikation in ihre Fahrzeuge integrieren wollen • Logistikunternehmen, die von optimierten Fahrzeiten auf städtischen Strecken profitieren könnten • Autonome Mobilitätsanbieter, die eine intelligente Spursteuerung in ihre Algorithmen integrieren möchten
Cost Structure • Entwicklungskosten für KI, Software und Hardware • Personalkosten für Entwickler, Projektleitung und Kundenbetreuung • Infrastrukturkosten für Server, Datenspeicherung und Cloudlösungen • Prototypenfertigung und Hardwarebeschaffung • Marketing, Schulung & Wartung • Reisekosten, Messen & Kommunikation mit Partnern		Revenue Streams • Lizenzierung der KI-Software an Kommunen oder Systemintegratoren • Verkauf von Kamera- und Weicheneinheiten als Komplettsystem • Datenanalyse & Verkehrsoptimierung als Dienstleistung (z. B. für Logistikunternehmen) • Abonnement-Modell für kontinuierliche KI-Optimierung und Systemupdates • Fördermittel & Innovationszuschüsse für die Markteinführung		

Abbildungen

Abb. 1 „Modellbau“

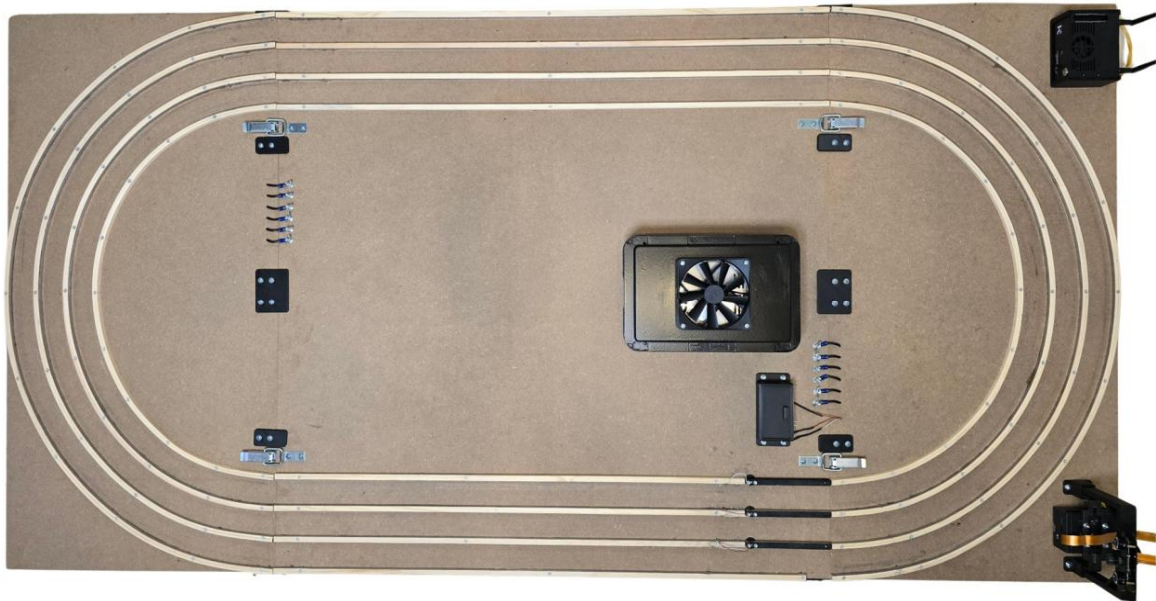


Abb. 2 „Motor-Getriebe-Einheit“

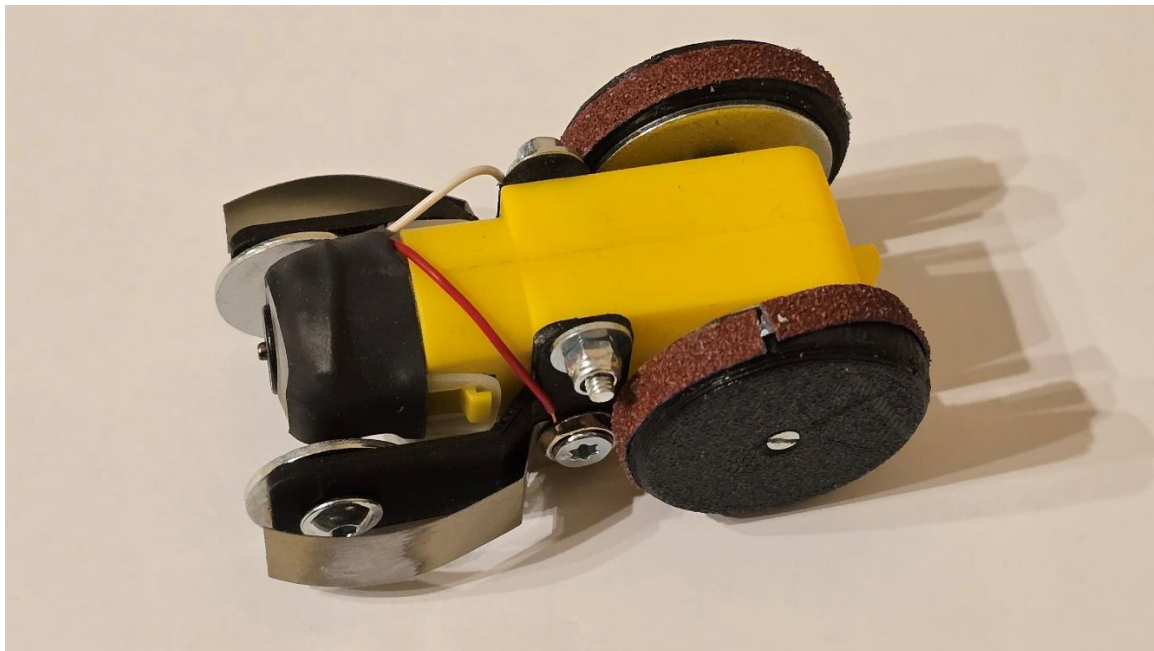


Abb. 3 „Leitplanken“

bending guardrail left

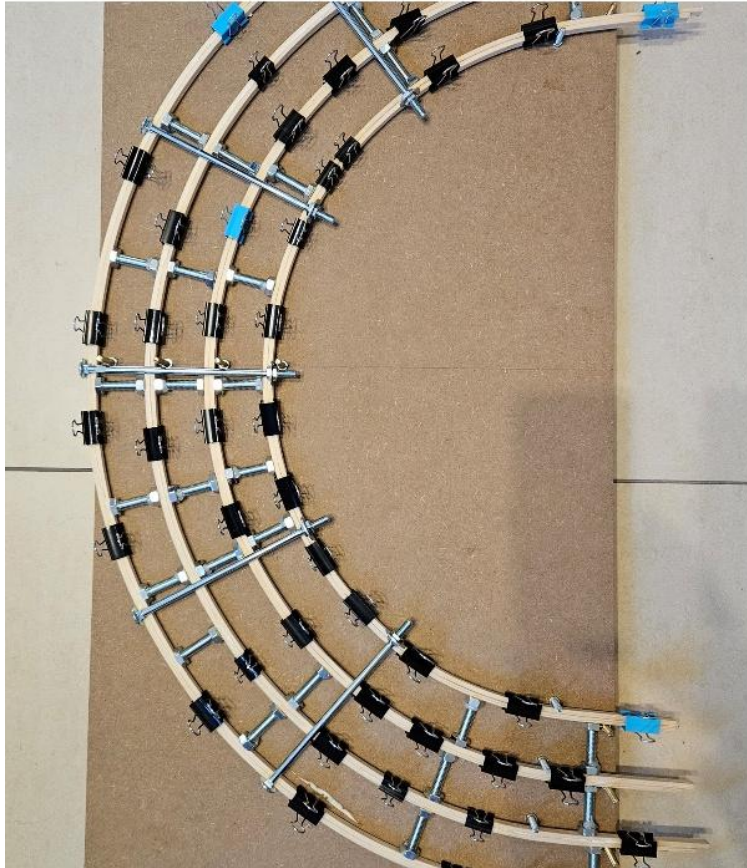


Abb. 4 „Netzteil & Stromversorgung“

power supply



Abb. 5.1 „Rad – Version 01“

tire - version 01

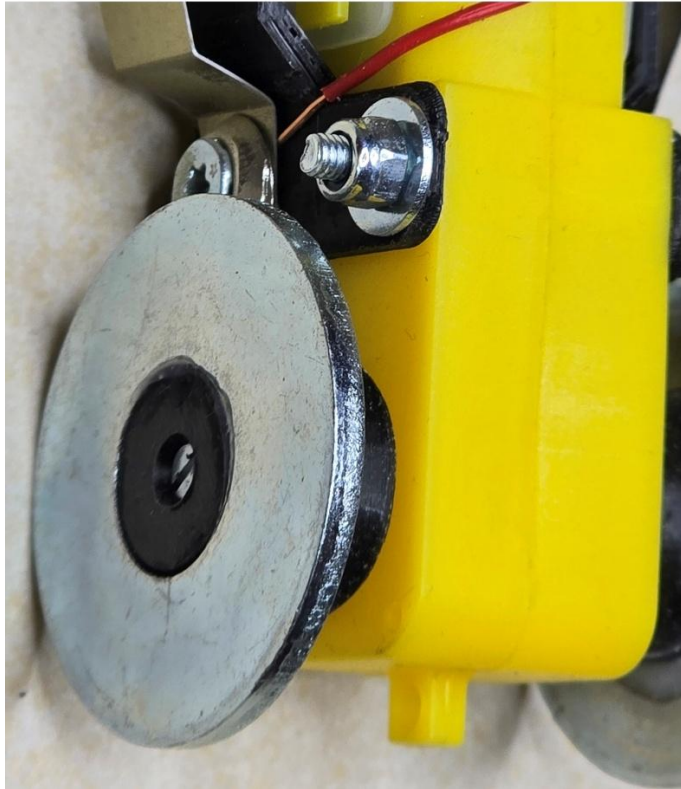


Abb. 5.2 „Rad – Version 05“

tire - version 05



Abb. 6 „Display – Visualisierung der Entscheidungen“



Abb. 7 „Schiebemechanismus“

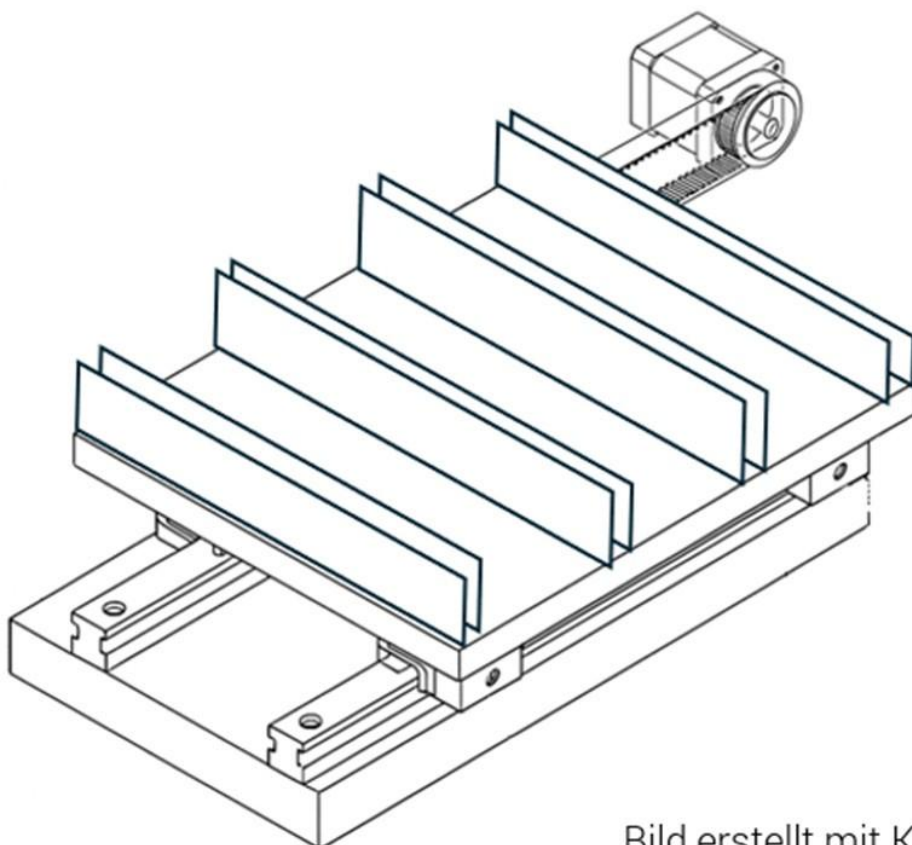


Abb. 8 „Schaltplan“

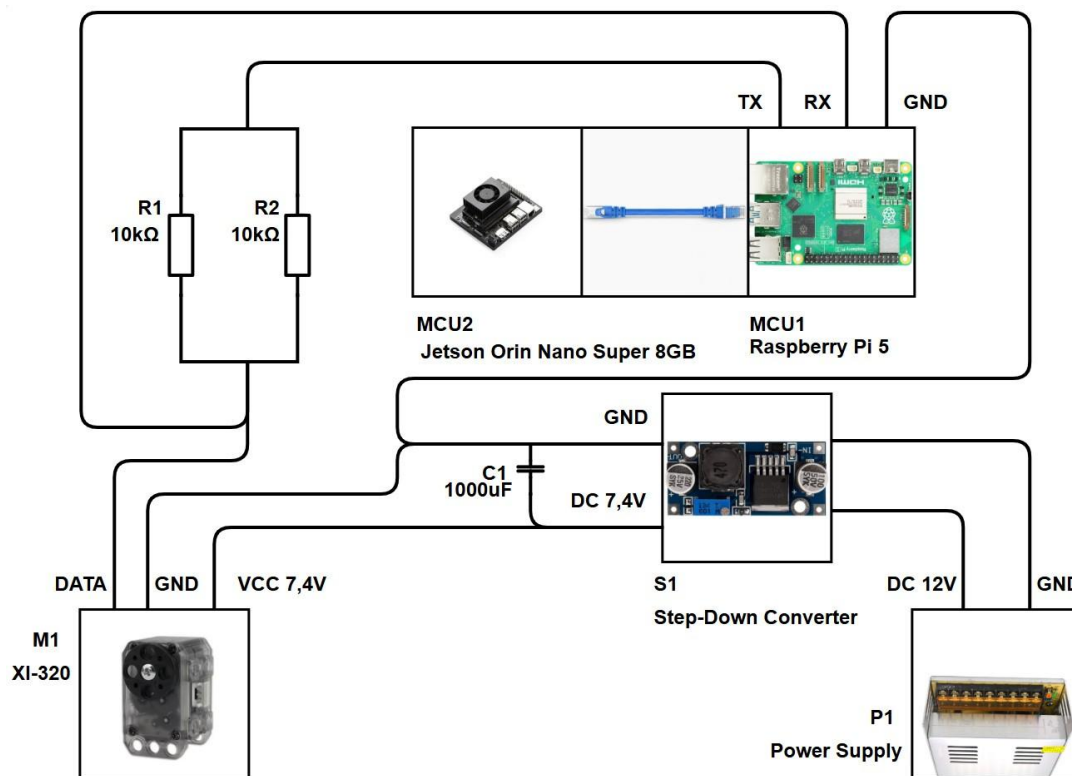


Abb. 9 „Schaubild – Nachrichtentechnik“

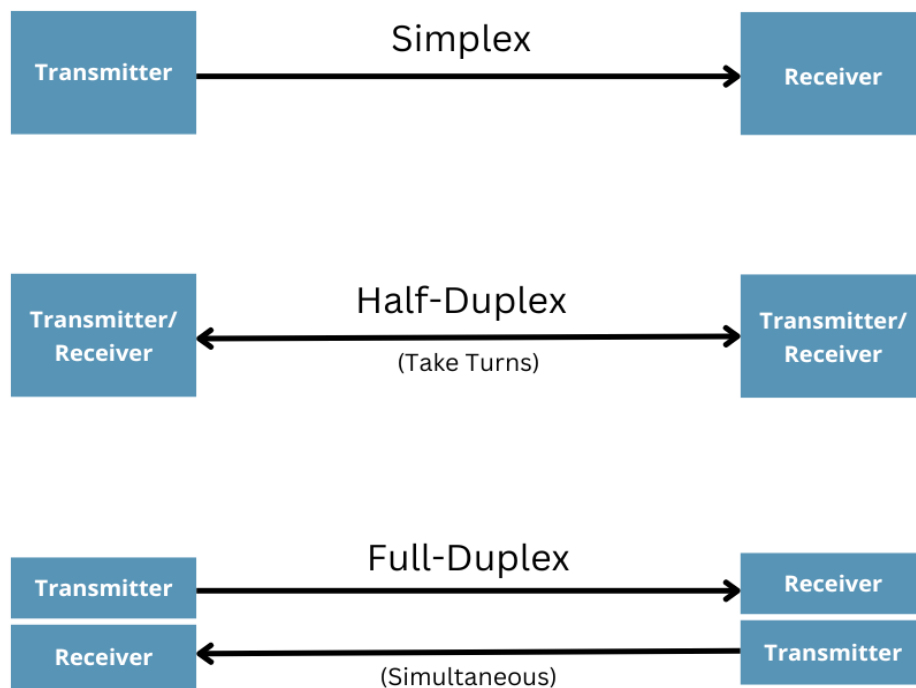


Abb. 10 „Fahrzeugerkennung mittels YOLOv11“

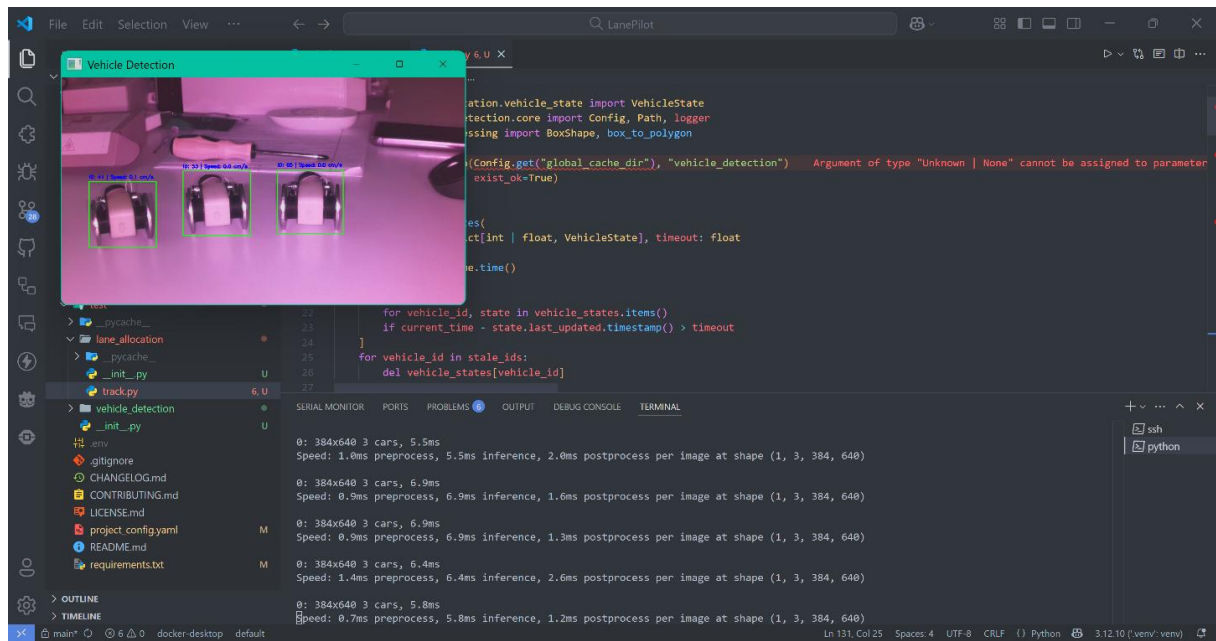


Abb. 11 „Beispielhafte graphische Darstellung eines Verkehrsszenarios“

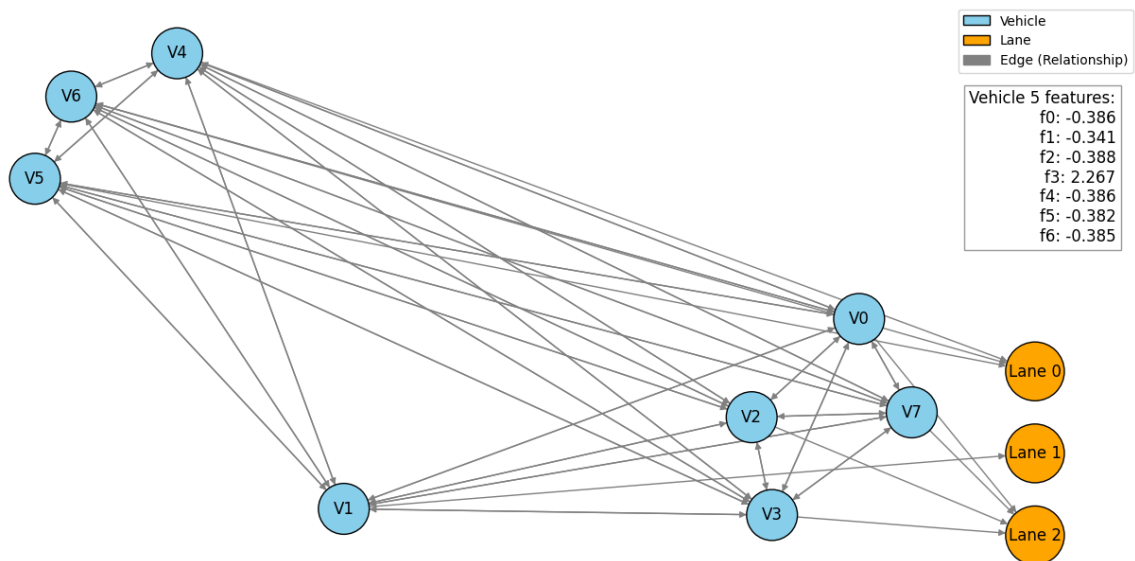


Abb. 12 „Graph Attention Network – Modell-Parameter“

```
1 dataset:
2   path: $ROOT_DIR/assets/data/lane_allocation/LanePilot.lane_allocation.zip
3
4 environment:
5   seed: 42 # Random seed for reproducibility
6   num_lanes: 3 # Number of lanes in the environment
7   max_vehicles_per_lane: 8 # Maximum number of vehicles per lane
8   normalization_mode: z_score # Normalization method for the dataset (options: min_max, z_score)
9
10 model:
11   num_epochs: 200 # Number of epochs for training
12   batch_size: 512 # Batch size for training
13   num_heads: 8 # Number of attention heads in the model
14   hidden_dim: 64 # Hidden dimension for the model
15
16 early_stopping:
17   patience: 20 # Number of epochs with no improvement after which training will be stopped
18
19 optimizer:
20   learning_rate: 0.003 # Learning rate for the model
21   t_0: 2 # Number of iterations for the cosine annealing schedule
22   t_mult: 4 # Multiplier for the cosine annealing schedule
23   weight_decay: 0.001 # Weight decay for the model
24   epsilon: 0.000001 # Epsilon for the optimizer
25
26 vehicle:
27   height_cm: 3.25
28   max_distance_cm: 0 # Distance between vehicles
29
30 camera:
31   resolution: [640, 384] # The camera resolution is 1280x720, but since the model was trained on 640x640 images with black infill,
32   # it is being cropped to 640x384.
33   fov_deg: 70
34
```